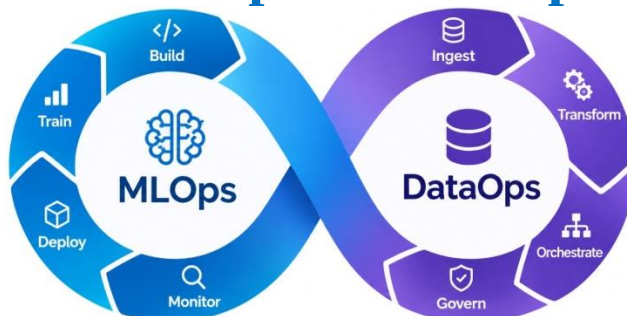


DÉPARTEMENT DE MATHÉMATIQUES ET INFORMATIQUE

Master d'excellence en IA et big data & data science

RAPPORT DE PROJET DE FIN DE MODULE :

MLOps & DataOps



Industrial Equipment Health Platform:

**Industrialisation d'une chaîne MLOps et DataOps pour la maintenance
prédictive d'équipements industriels**

Réalisé par :

Oussama Ashad

Mohamed Akrame Kraichy

Hamza El Haddaji

Mouhcine Ayar

Hajar En-Najdy

Aya Moujoud

Mohamed Amine Kar-any

Iliass Ouchida

Exposé le 18 Septembre 2026 devant les membres de jury :

Pr. Mohammed Aitdaoud

Faculté des sciences Ben M'Sick

Encadrant

Pr. Taoufik Amzil

Faculté des sciences Ben M'Sick

Co-Encadrant

Année Universitaire : 2025 - 2026

Remerciements

Nous tenons à exprimer notre gratitude à l'ensemble du corps enseignant du Master Intelligence Artificielle de la Faculté des Sciences Ben M'Sik pour la qualité de la formation dispensée et pour l'exigence méthodologique qui a guidé ce travail.

Nos remerciements s'adressent en particulier à l'encadrant du module MLOps S DataOps, dont les orientations ont permis de recentrer le projet sur son objectif essentiel : non pas entraîner un modèle de plus, mais industrialiser le cycle de vie complet d'un produit de données. L'autorisation accordée de réaliser le déploiement final en local, via Docker Compose, a également été déterminante pour la faisabilité du projet dans les délais impartis.

Nous remercions enfin le NASA Ames Prognostics Center of Excellence, qui met à disposition de la communauté scientifique le jeu de données Turbofan Engine Degradation Simulation (C-MAPSS), socle empirique de l'ensemble de nos expérimentations.

Chaque membre de l'équipe a contribué à ce travail dans son périmètre propre, et les difficultés rencontrées notamment en matière d'intégration de branches et de coordination ont constitué, rétrospectivement, la partie la plus formatrice de l'exercice.

Résumé

Ce rapport présente la conception, la réalisation et l'évaluation d'une plateforme complète de maintenance prédictive industrielle, baptisée Industrial Equipment Health Platform. Le projet ne se limite pas à la construction d'un modèle d'apprentissage automatique : il vise l'industrialisation de la totalité du cycle de vie d'un produit de données, de l'ingestion des fichiers bruts jusqu'au monitoring des prédictions en production, en passant par le stockage analytique, la transformation, les tests de qualité, l'orchestration, le suivi d'expériences, le registre de modèles, l'exposition par API, la conteneurisation et l'intégration continue.

Le cas d'usage retenu est l'estimation de la durée de vie résiduelle (Remaining Useful Life, RUL) de turboréacteurs à partir de mesures multivariées de capteurs. Le jeu de données utilisé est le jeu de référence NASA C-MAPSS, dans ses quatre sous-ensembles FD001 à FD004, représentant 160 359 lignes d'entraînement et 104 897 lignes de test pour 709 et 707 moteurs respectivement.

La chaîne DataOps repose sur dlt pour l'ingestion, DuckDB comme entrepôt analytique local, dbt pour les transformations et les tests de qualité, et Dagster pour l'orchestration des actifs.

La chaîne MLOps repose sur scikit-learn pour la modélisation, MLflow pour le suivi et le registre de modèles, FastAPI pour le service de prédiction, Docker et Docker Compose pour la reproductibilité, et GitHub Actions pour l'intégration continue.

Le modèle final retenu est un HistGradientBoostingRegressor entraîné sur 89 variables, dont des statistiques glissantes sur cinq cycles (moyenne, écart-type, pente) calculées de manière causale. Évalué selon le protocole standard du dernier cycle observé par moteur, il atteint une erreur absolue moyenne de 12,56 cycles et une racine de l'erreur quadratique moyenne de 16,93 cycles, contre 37,32 et 44,44 pour une référence naïve à la moyenne, soit une réduction d'environ 66 % de l'erreur absolue.

Au-delà des performances prédictives, la contribution principale du travail réside dans la démonstration d'une chaîne reproductible en une commande, d'une gouvernance explicite de la qualité des données par contrats et tests dbt, et d'une discipline de traçabilité couvrant les versions de modèle, les journaux de prédiction et la détection de dérive.

Abstract

This report presents the design, implementation and evaluation of a complete industrial predictive-maintenance platform, the Industrial Equipment Health Platform. The project is not limited to building a machine-learning model: it aims to industrialise the entire lifecycle of a data product, from raw file ingestion to production prediction monitoring, including analytical storage, transformation, quality testing, orchestration, experiment tracking, model registry, API serving, containerisation and continuous integration.

The selected use case is the estimation of the Remaining Useful Life (RUL) of turbofan engines from multivariate sensor measurements. The dataset is the reference NASA C-MAPSS collection, using all four subsets FD001 to FD004, amounting to 160,359 training rows and 104,897 test rows covering 709 and 707 engines respectively.

The DataOps chain relies on dlt for ingestion, DuckDB as a local analytical warehouse, dbt for transformations and data-quality tests, and Dagster for asset orchestration. The MLOps chain relies on scikit-learn for modelling, MLflow for tracking and model registry, FastAPI for prediction serving, Docker and Docker Compose for reproducibility, and GitHub Actions for continuous integration.

The final model is a HistGradientBoostingRegressor trained on 89 features, including causally computed five-cycle rolling statistics (mean, standard deviation, slope). Evaluated with the standard last-observed-cycle protocol per engine, it reaches a mean absolute error of 12.56 cycles and a root mean squared error of 16.93 cycles, compared with 37.32 and 44.44 for a naive mean baseline, a reduction of roughly 66 % in absolute error.

Beyond predictive performance, the main contribution of this work is the demonstration of a one-command reproducible pipeline, an explicit data-quality governance framework based on contracts and dbt tests, and a traceability discipline covering model versions, prediction logs and drift detection.

Table des matières

Remerciements	2
Résumé	3
Abstract	4
Liste des tableaux	9
Liste des figures	10
Introduction générale	11
1.1. Contexte industriel	11
1.2. Problématique	11
1.3. Objectifs du projet	12
1.4. Périmètre et hypothèses de travail	13
1.5. Démarche méthodologique adoptée	14
1.6. Contributions du travail	14
1.7. Organisation du rapport	14
État de l'art et cadre théorique	16
2.1. Le pronostic et la gestion de la santé des systèmes	16
2.2. Formalisation de la durée de vie résiduelle	16
2.3. Métriques d'évaluation du pronostic	17
2.4. Du modèle au système : l'émergence du MLOps	17
2.5. Le DataOps et l'architecture en couches	18
2.6. Contrats de données et qualité	19
2.7. La dérive et la surveillance après déploiement	19
2.8. Synthèse et positionnement	19
Cadre méthodologique et gestion de projet	21
3.1. Choix d'un cadre agile	21
3.2. Organisation de l'équipe et répartition des responsabilités	21
3.3. Le carnet de produit	21
3.4. Déroulement des sprints	22
3.5. Définition de « terminé »	23
3.6. Analyse critique de la conduite de projet	23
Le jeu de données NASA C-MAPSS et son analyse	24
4.1. Origine et nature des données	24
4.2. Structure des fichiers	24
4.3. Volumétrie effective	25

4.4.	Analyse exploratoire	25
4.5.	Licence et gouvernance des données	26
Architecture de la plateforme.....		28
5.1.	Principes directeurs	28
5.2.	Vue d'ensemble du flux	28
5.3.	Justification des choix technologiques.....	29
5.4.	Modèle en couches de l'entrepôt	29
5.5.	Architecture de service	30
5.6.	Une console d'ingénierie comme surface de démonstration.....	30
La chaîne DataOps : ingestion, stockage, transformation, orchestration.....		32
6.1.	L'ingestion avec dlt.....	32
6.2.	DuckDB comme entrepôt analytique local	32
6.3.	Les transformations dbt	33
6.4.	Orchestration avec Dagster	34
6.5.	Exécution locale et vérification	34
Gouvernance de la qualité : contrats, tests et lignage		36
7.1.	Pourquoi une gouvernance explicite.....	36
7.2.	Les contrats de données.....	36
7.3.	Les six dimensions de la qualité.....	36
7.4.	Les tests dbt effectivement implémentés	37
7.5.	Le lignage des données.....	38
7.6.	La stratégie de prévention des fuites d'information	38
Ingénierie des variables explicatives et modélisation		40
8.1.	Formulation du problème d'apprentissage.....	40
8.2.	Construction des variables explicatives	40
8.3.	Protocole expérimental.....	41
8.4.	Le modèle final	41
8.5.	Validation croisée par groupes.....	42
8.6.	Chaîne d'entraînement et reproductibilité	42
Couche MLOps : suivi, registre et service de prédiction		44
9.1.	Le suivi des expériences avec MLflow	44
9.2.	Le registre de modèles.....	44
9.3.	Le service de prédiction FastAPI	45
9.4.	La console d'ingénierie et ses points d'entrée étendus	46
Conteneurisation, intégration continue et surveillance		47
10.1.	La reproductibilité comme exigence première	47
10.2.	Conception des images et de la pile.....	47

10.3.	La chaîne d'intégration continue.....	48
10.4.	Stratégie de tests.....	48
10.5.	Surveillance et observabilité	49
10.6.	Le rapport de disponibilité	50
Résultats et discussion.....		51
11.1.	Résultats comparatifs des modèles	51
11.2.	Lecture incrémentale des gains	52
11.3.	Analyse des deux protocoles d'évaluation.....	52
11.4.	Cohérence entre validation et test.....	53
11.5.	Interprétation opérationnelle des résultats	53
11.6.	Résultats de la chaîne DataOps et d'industrialisation.....	54
11.7.	Discussion critique	54
Limites et perspectives		56
12.1.	Limites liées aux données	56
12.2.	Limites liées à la modélisation	56
12.3.	Limites liées à l'industrialisation	56
12.4.	Perspectives d'évolution.....	57
Conclusion générale		59
Bibliographie et sources.....		61
Jeu de données.....		61
Pronostic et maintenance prédictive.....		61
MLOps, DataOps et dette technique		61
Documentation technique des outils		61
Artefacts internes du projet		62

Liste des abréviations

Sigle	Signification
API	Application Programming Interface / interface de programmation applicative
CI/CD	Continuous Integration / Continuous Delivery /intégration et livraison continues
C-MAPSS	Commercial Modular Aero-Propulsion System Simulation
CBM	Condition-Based Maintenance / maintenance conditionnelle
dbt	data build tool / outil de transformation analytique déclarative
dlt	data load tool / bibliothèque Python d'ingestion de données
DAG	Directed Acyclic Graph / graphe orienté acyclique
EDA	Exploratory Data Analysis /analyse exploratoire des données
ELT	Extract, Load, Transform
ETL	Extract, Transform, Load
HGB	HistGradientBoostingRegressor / régresseur à gradient boosté par histogrammes
JSONL	JSON Lines / format de journalisation ligne à ligne
MAE	Mean Absolute Error / erreur absolue moyenne
ML	Machine Learning / apprentissage automatique
MLOps	Machine Learning Operations
PHM	Prognostics and Health Management
RMSE	Root Mean Squared Error / racine de l'erreur quadratique moyenne
RUL	Remaining Useful Life / durée de vie résiduelle
SLA	Service Level Agreement / engagement de niveau de service
SQL	Structured Query Language

Liste des tableaux

Table 1	Organisation générale du rapport et correspondance avec les livrables du dé.....	15
Table 2	Positionnement du projet sur une échelle de maturité MLOps.	18
Table 3	Carnet de produit hiérarchisé et critères d'acceptation.	22
Table 4	Caractéristiques des quatre sous-ensembles C-MAPSS.	24
Table 5	Volumétrie effective de l'exécution finale.	25
Table 6	Choix technologiques et justifications.	29
Table 7	Organisation en schémas de l'entrepôt DuckDB.	29
Table 8	Services de la pile d'exécution.	30
Table 9	Les sept actifs du graphe d'orchestration Dagster.	34
Table 10	Résultats de l'exécution locale de la chaîne DataOps.	35
Table 11	Contrats de données et responsabilités associées.	36
Table 12	Tests de qualité dbt exécutés à chaque reconstruction (8 tests, 8 réussis).	38
Table 13	Composition du vecteur de 89 variables explicatives.	40
Table 14	Les cinq modèles du protocole comparatif.	41
Table 15	Résultats de la validation croisée à cinq plis par groupes de moteurs.	42
Table 16	Paramètres de gouvernance du registre de modèles.	44
Table 17	Structure de la réponse du point d'entrée de prédiction.	45
Table 18	point d'entrée de consol.	46
Table 19	Volumes persistants de la pile Docker Compose.	47
Table 20	Les dix étapes de la chaîne d'intégration continue.	48
Table 21	Couverture de la stratégie de tests.	49
Table 22	Signaux de surveillance et leur implémentation.	49
Table 23	Exemple de rapport de dérive généré durant la répétition.	50
Table 24	Performances comparées selon le protocole standard du dernier cycle (707 moteurs).	51
Table 25	Décomposition incrémentale des gains de performance.	52
Table 26	Comparaison des deux protocoles d'évaluation pour le modèle final.	52
Table 27	Cohérence entre validation croisée interne et évaluation externe.	53
Table 28	Synthèse des résultats d'industrialisation vérifiés.	54
Table 29	Perspectives à court terme.	57

Liste des figures

Figure 1— Chaîne de traitement de bout en bout de la plateforme.	28
Figure 2 — Vue de la console d'ingénierie : observatoire de dégradation d'un turboréacteur.	31
Figure 3— Lignage effectif de la donnée brute jusqu'aux journaux de prédiction.	38
Figure 4— Évaluation du modèle amélioré : distribution des erreurs et adéquation prédiction/réalité.	43
Figure 5— Comparaison graphique des cinq modèles évalués.	51

CHAPITRE 1

Introduction générale

1.1. Contexte industriel

La maintenance des équipements industriels représente un coût important pour de nombreux secteurs, notamment l'industrie, l'énergie et le transport. Dans le domaine aéronautique, une panne imprévue d'un équipement important, comme un turboréacteur, peut avoir de nombreuses conséquences : arrêt de l'appareil, perturbation des vols, coûts supplémentaires et risques pour la sécurité.

Il existe principalement trois types de maintenance. La **maintenance corrective** consiste à intervenir après l'apparition d'une panne. Elle peut être simple à mettre en place, mais elle présente un risque important de panne imprévue. La **maintenance préventive** consiste à effectuer des interventions à des intervalles définis à l'avance. Cette méthode permet de réduire les risques, mais peut entraîner le remplacement de pièces qui fonctionnent encore correctement. Enfin, la **maintenance conditionnelle** utilise les informations provenant des capteurs pour surveiller l'état réel de l'équipement et décider quand une intervention est nécessaire.

La **maintenance prédictive** va plus loin en utilisant les données des capteurs et les techniques d'intelligence artificielle pour prévoir les pannes avant qu'elles ne se produisent. Elle permet notamment d'estimer la **durée de vie restante (RUL – Remaining Useful Life)** d'un équipement, c'est-à-dire le nombre de cycles ou d'heures de fonctionnement qu'il peut encore effectuer avant une éventuelle défaillance.

Avec le développement des capteurs, du stockage des données et de l'apprentissage automatique, la maintenance prédictive devient de plus en plus accessible. Cependant, créer un modèle d'intelligence artificielle ne suffit pas pour répondre aux besoins d'une entreprise. Le modèle doit être intégré dans une chaîne complète permettant de traiter les données, entraîner et déployer le modèle, suivre ses performances et détecter les problèmes.

C'est dans ce contexte que s'inscrit notre projet **Industrial Equipment Health Platform**, qui vise à mettre en place une plateforme complète de maintenance prédictive, depuis la collecte des données jusqu'au suivi des prédictions en production.

1.2. Problématique

La littérature professionnelle rapporte de manière constante qu'une large majorité des projets d'apprentissage automatique n'atteignent jamais la production. Les causes identifiées sont rarement d'ordre algorithmique : elles relèvent de l'absence de reproductibilité, de la dégradation silencieuse de la qualité des données, de l'écart entre l'environnement d'expérimentation et l'environnement de service, de l'impossibilité de tracer la version de modèle ayant produit une décision donnée, et de l'absence de surveillance après déploiement.

La problématique de ce projet peut donc être formulée ainsi :

Comment concevoir une plateforme de maintenance prédictive fiable, reproductible et facilement déployable, permettant de gérer les données, d'entraîner et d'évaluer les modèles, de les déployer en production et de suivre leurs performances ?

Pour répondre à cette problématique, plusieurs questions sont étudiées :

1. **Comment organiser les données ?**
Comment séparer les données brutes, les données transformées et les données prêtes à être utilisées afin de faciliter leur gestion et de réaliser des contrôles de qualité à chaque étape ?
2. **Comment garantir la qualité des données ?**
Comment définir des règles et des contrôles permettant de vérifier automatiquement que les données respectent les attentes du système ?
3. **Comment évaluer correctement le modèle ?**
Comment mesurer les performances du modèle tout en évitant les fuites de données et en utilisant des références permettant de comparer les résultats ?
4. **Comment garantir le bon fonctionnement du modèle en production ?**
Comment s'assurer que le service de prédiction utilise réellement le modèle entraîné et la bonne version du modèle ?
5. **Comment rendre la plateforme reproductible ?**
Comment permettre à une autre personne de reproduire l'ensemble du processus, depuis les données jusqu'au déploiement, de manière simple et fiable ?

1.3. Objectifs du projet

1.3.1. Objectif général

Construire une plateforme de santé des équipements industriels qui prédit le risque de dégradation et la durée de vie résiduelle à partir de mesures multivariées de capteurs, et qui expose cette prédiction à travers une interface de programmation de qualité production.

1.3.2. Objectifs spécifiques

- **Objectif DataOps** : mettre en place une chaîne d'ingestion et de transformation versionnée, testée et orchestrée, produisant une table de variables explicatives prête pour l'apprentissage, à partir des fichiers bruts NASA C-MAPSS.
- **Objectif Qualité** : formaliser des contrats de données pour chaque couche de l'entrepôt, les faire appliquer par des tests automatisés, et documenter le lignage complet de la donnée brute jusqu'à la prédiction.
- **Objectif Modélisation** : entraîner un modèle de régression de la RUL comparé à des références explicites, avec une validation croisée respectant les frontières entre moteurs et une stratégie documentée de prévention des fuites d'information.
- **Objectif MLOps** : suivre chaque exécution d'entraînement dans MLflow, enregistrer le modèle dans un registre avec alias de promotion, et servir ce modèle exact via FastAPI sans possibilité de repli silencieux.
- **Objectif Exploitation** : conteneuriser l'ensemble, automatiser les contrôles par intégration continue, journaliser chaque prédiction et produire un rapport de dérive exploitable.

1.4. Périmètre et hypothèses de travail

Périmètre du projet

Le projet se limite volontairement à la conception d'une chaîne complète de maintenance prédictive fonctionnant dans un environnement local. Conformément aux consignes du projet, le déploiement est réalisé localement à l'aide de Docker Compose, sans utiliser une infrastructure cloud.

Cette approche met l'accent sur la reproductibilité. L'objectif est de permettre à un autre utilisateur ou évaluateur de reproduire l'ensemble du projet sur son propre ordinateur, avec les mêmes étapes et les mêmes résultats.

Hypothèses du projet

Le travail repose sur trois principales hypothèses.

Premièrement, les données C-MAPSS, bien qu'elles soient simulées, sont utilisées comme représentation d'un flux de données provenant d'équipements industriels. Elles permettent notamment de travailler avec des séries temporelles contenant plusieurs variables, différentes conditions de fonctionnement et des informations sur l'état des équipements.

Deuxièmement, la durée de vie restante (RUL – Remaining Useful Life) est considérée comme une valeur continue. La valeur maximale de la RUL est limitée à 125 cycles, conformément à l'approche couramment utilisée avec les données C-MAPSS.

Enfin, le niveau de risque présenté à l'utilisateur est déterminé à partir de la RUL prédite. Des seuils sont appliqués à cette valeur afin d'identifier différents niveaux de risque. Le projet n'utilise donc pas un modèle de classification indépendant pour déterminer le niveau de risque. Cette simplification est prise en compte dans l'analyse des limites du projet.

1.5. Démarche méthodologique adoptée

La démarche suivie dans ce projet combine une approche d'ingénierie logicielle et une approche expérimentale.

Sur le plan logiciel, le projet a été organisé selon une méthode agile composée de trois sprints. Chaque sprint s'appuie sur un ensemble de tâches définies à l'avance, avec des critères permettant de vérifier que les objectifs ont été atteints. Des revues ont également été réalisées à la fin de chaque sprint afin de suivre l'avancement du projet.

Sur le plan expérimental, les résultats obtenus sont associés à des éléments vérifiables présents dans le projet. Il peut s'agir, par exemple, de fichiers contenant les métriques au format JSON, de notebooks Jupyter exécutés pour réaliser les expérimentations ou encore de rapports permettant de suivre l'évolution des performances du système.

Cette démarche repose sur un principe simple : chaque résultat présenté doit pouvoir être vérifié et reproduit. Cela permet d'améliorer la fiabilité du projet et de faciliter son évaluation par une autre personne.

Ce principe est également appliqué au service de prédiction. Le système ne doit pas produire une prédiction lorsque le modèle attendu n'a pas été correctement chargé. Dans ce cas, il est préférable de signaler clairement l'erreur plutôt que de retourner une valeur qui pourrait sembler correcte mais qui ne provient pas réellement du modèle entraîné.

1.6. Contributions du travail

Les contributions revendiquées sont les suivantes :

- Une architecture en couches complète, documentée et exécutable, couvrant sept étapes du cycle de vie d'un produit de données.
- Une table de variables explicatives construite intégralement en dbt, garantissant que la logique de transformation est versionnée, testée et lisible, plutôt que dissimulée dans un script Python ad hoc.
- Une évaluation comparative de cinq familles de modèles sur les quatre sous-ensembles C-MAPSS simultanément, ce qui est plus exigeant que l'usage fréquent limité au seul FD001.
- Un protocole d'évaluation double : métrique standard du dernier cycle observé par moteur, et métrique en ligne au niveau de la ligne, chacune étant explicitement qualifiée quant à son statut probant.
- Une chaîne d'intégration continue qui ne se contente pas de vérifier la syntaxe mais rejoue la totalité du flux, y compris l'entraînement, la construction des conteneurs et un test de fumée sur la prédiction.

1.7. Organisation du rapport

Le présent rapport est organisé en douze chapitres. Le chapitre 2 dresse un état de l'art de la maintenance prédictive, du pronostic de durée de vie résiduelle et des pratiques MLOps et DataOps. Le chapitre 3 expose le cadre méthodologique et la gestion de projet agile. Le chapitre 4 décrit le jeu de données C-MAPSS et l'analyse exploratoire conduite. Le chapitre 5 présente l'architecture générale de la plateforme et les choix technologiques associés.

Les chapitres 6 et 7 détaillent respectivement la chaîne DataOps et la gouvernance de la qualité

des données. Le chapitre 8 est consacré à l'ingénierie des variables explicatives et à la modélisation. Le chapitre 9 traite de la couche MLOps proprement dite : suivi d'expériences, registre de modèles et service de prédiction. Le chapitre 10 aborde la conteneurisation, l'intégration continue et la surveillance. Le chapitre 11 présente et discute les résultats obtenus. Le chapitre 12 énonce les limites du travail et les perspectives d'extension. Une conclusion générale, une bibliographie et un ensemble d'annexes techniques closent le document.

Chapitre	Question traitée	Livrable associé
2	Que sait-on déjà du pronostic de RUL et du MLOps ?	Revue de littérature
3	Comment le travail a-t-il été organisé ?	Carnet de produit, plans de sprint
4	Sur quelles données travaille-t-on ?	Analyse exploratoire
5	Quelle est l'architecture cible ?	Schéma d'architecture, matrice de responsabilités
6	Comment la donnée est-elle ingérée et transformée ?	Chaîne dlt / DuckDB / dbt / Dagster
7	Comment la qualité est-elle garantie ?	Contrats, tests, lignage
8	Comment le modèle est-il construit et évalué ?	Rapport d'évaluation, carnet exécuté
9	Comment le modèle est-il suivi et servi ?	MLflow, service FastAPI
10	Comment l'ensemble est-il exploité ?	Docker Compose, CI, monitoring
11	Quels résultats ont été obtenus ?	Tableaux comparatifs, figures
12	Quelles sont les limites ?	Analyse critique, feuille de route

Table 1 Organisation générale du rapport et correspondance avec les livrables du dé

CHAPITRE 2

État de l'art et cadre théorique

2.1. Le pronostic et la gestion de la santé des systèmes

Le champ disciplinaire dans lequel s'inscrit ce projet est désigné dans la littérature anglo-saxonne par l'acronyme PHM, pour Prognostics and Health Management. Il s'agit d'un domaine à l'intersection de la fiabilité des systèmes, du traitement du signal et de l'apprentissage statistique, dont l'objectif est de diagnostiquer l'état de santé courant d'un système et de pronostiquer son évolution future.

La communauté PHM distingue classiquement trois familles d'approches. Les approches fondées sur des modèles physiques reposent sur une modélisation explicite des mécanismes de dégradation : propagation de fissures, fatigue thermomécanique, usure de paliers. Elles offrent une excellente interprétabilité et une bonne extrapolation hors du domaine observé, mais exigent une connaissance fine de la physique du système et un effort d'instrumentation et de calibration considérable.

Les approches fondées sur les données infèrent la dégradation directement à partir de l'historique des mesures, sans modèle physique explicite. Elles sont beaucoup plus rapides à déployer dès lors qu'un historique instrumenté existe, mais restent captives du domaine couvert par les données d'apprentissage et souffrent d'une interprétabilité limitée. Les approches hybrides, enfin, combinent une base physique avec une correction apprise ; elles représentent la direction la plus active de la recherche contemporaine mais supposent la disponibilité simultanée des deux types de ressources.

Le présent projet s'inscrit résolument dans la deuxième famille, ce qui correspond à la nature du jeu de données disponible et à la vocation pédagogique du module : l'objet d'étude n'est pas la physique du turboréacteur mais l'industrialisation de la chaîne de traitement.

2.2. Formalisation de la durée de vie résiduelle

Soit un équipement indexé par i observé à des instants discrets t appelés cycles. La durée de vie résiduelle au cycle t est définie, pour une unité dont la défaillance survient au cycle T , par la relation :

$$RUL_i(t) = T_i - t$$

Dans les données d'entraînement C-MAPSS, chaque trajectoire est observée jusqu'à la défaillance, de sorte que T est directement égal au cycle maximal observé pour l'unité. La cible est donc calculable sans ambiguïté. Dans les données de test, en revanche, la trajectoire est tronquée avant la défaillance et la RUL réelle au dernier cycle observé est fournie séparément dans un fichier d'étiquettes ; la RUL en tout point de la trajectoire de test s'obtient alors comme la somme de cette valeur et du nombre de cycles restant à parcourir jusqu'au dernier cycle observé.

2.2.1. L'écèlement de la cible

Une pratique méthodologique largement adoptée dans la littérature C-MAPSS consiste à écèlement la RUL à une valeur maximale, typiquement 125 ou 130 cycles. La justification est double. D'une part, en début de vie, les signaux de capteurs sont quasiment stationnaires : la dégradation n'a pas encore produit de signature exploitable, et la valeur exacte de la RUL est donc intrinsèquement imprédictible. D'autre part, l'écèlement évite que la fonction de perte ne soit dominée par des erreurs commises sur des périodes où la prédiction n'a de toute façon aucune utilité opérationnelle : un opérateur de maintenance ne prend aucune décision différente selon qu'il reste 200 ou 250 cycles.

Dans ce projet, le seuil d'écèlement a été fixé à 125 cycles avant toute observation des performances de test, ce qui est une condition nécessaire pour que le protocole reste honnête. Fixer ce seuil après avoir observé les résultats reviendrait à une forme d'optimisation sur le jeu de test.

2.3. Métriques d'évaluation du pronostic

2.3.1. Erreur absolue moyenne et erreur quadratique

L'erreur absolue moyenne (MAE) mesure l'écart moyen en valeur absolue entre RUL prédite et RUL réelle. Elle a l'avantage d'être directement interprétable dans l'unité du problème : une MAE de 12,5 signifie que la prédiction se trompe en moyenne d'environ douze cycles et demi. La racine de l'erreur quadratique moyenne (RMSE) pénalise davantage les erreurs importantes ; l'écart entre MAE et RMSE renseigne donc sur la dispersion des erreurs et sur la présence d'observations aberrantes.

2.3.2. Le score asymétrique NASA

La métrique spécifique proposée avec le jeu de données C-MAPSS traduit une asymétrie opérationnelle fondamentale : **surestimer la durée de vie résiduelle est plus grave que la sous-estimer**. Une sous-estimation conduit à une maintenance anticipée, donc à un coût économique ; une surestimation conduit à une défaillance en service, donc à un risque de sécurité. Le score pénalise donc exponentiellement les erreurs positives plus fortement que les erreurs négatives.

Cette métrique présente une propriété importante à connaître pour interpréter les résultats : parce qu'elle est exponentielle et sommée, sa valeur dépend directement du nombre d'observations évaluées. Un score calculé sur 707 moteurs et un score calculé sur 102 069 lignes ne sont pas comparables entre eux, même s'ils portent sur le même modèle. Cette propriété explique les écarts d'ordre de grandeur observés au chapitre 11 et justifie que la métrique du dernier cycle soit retenue comme métrique de référence.

2.3.3. Protocole d'évaluation retenu

Deux protocoles sont utilisés en parallèle dans ce travail. Le **protocole standard** évalue une seule prédiction par moteur de test, au dernier cycle observé ; c'est le protocole de référence de la littérature C-MAPSS et il constitue la métrique probante du projet. Le **protocole en ligne** évalue une prédiction à chaque cycle de chaque trajectoire de test ; il reflète mieux l'usage réel d'un service de prédiction interrogé en continu, mais il n'est pas comparable aux résultats publiés. Les deux sont rapportés, avec une hiérarchie explicite entre eux.

2.4. Du modèle au système : l'émergence du MLOps

Le constat fondateur du MLOps est que le code d'apprentissage automatique ne représente qu'une fraction minime d'un système d'apprentissage automatique réel. L'essentiel de la

complexité réside dans la collecte des données, la vérification de leur qualité, la gestion des ressources de calcul, l'extraction des variables explicatives, l'analyse des processus, la gestion de la configuration, l'infrastructure de service et les outils de surveillance.

Cette asymétrie engendre une forme spécifique de dette technique. Un système d'apprentissage automatique accumule en effet des dettes que le génie logiciel classique ne connaît pas : l'enchevêtrement des variables explicatives, où modifier une entrée modifie l'importance de toutes les autres ; les boucles de rétroaction cachées, où les prédictions du modèle influencent les données futures ; les dépendances de données instables, plus difficiles à détecter que les dépendances de code car aucun compilateur ne les signale.

Le MLOps répond à cette situation par un ensemble de pratiques : versionnement conjoint du code, des données et des modèles ; automatisation des chaînes d'entraînement ; suivi systématique des expériences ; registre de modèles avec états de promotion ; tests automatisés portant à la fois sur le code et sur les données ; et surveillance continue après déploiement.

Niveau de maturité	Caractéristique dominante	Situation du projet
Niveau 0 _manuel	Entraînement et déploiement manuels, carnets isolés, aucune reproductibilité	Dépassé dès le sprint 1
Niveau 1 _chaîne automatisée	Entraînement orchestré, artefacts versionnés, tests de données	Atteint et documenté
Niveau 2 _CI/CD complet	Reconstruction et redéploiement automatiques déclenchés par le code ou la donnée	Atteint pour le code ; partiel pour le réentraînement déclenché par la dérive
Niveau 3 _boucle fermée	Réentraînement automatique piloté par la dérive détectée, promotion automatisée	Non atteint ,perspective documentée

Table 2Positionnement du projet sur une échelle de maturité MLOps.

2.5. Le DataOps et l'architecture en couches

Le DataOps applique au domaine de la donnée les principes du DevOps : automatisation, tests, intégration continue et collaboration. Sa pratique structurante est l'organisation de l'entrepôt en couches successives, chacune ayant un contrat et un propriétaire.

- La couche brute est une copie fidèle et immuable de la source. Aucune transformation métier n'y est appliquée, ce qui garantit la possibilité de tout reconstruire en cas d'erreur de logique en aval.
- La couche intermédiaire ou staging applique le nettoyage technique : typage, déduplication, normalisation des noms, filtrage des lignes structurellement invalides.
- La couche exposée ou marts contient les tables métier, ici la table de variables explicatives consommée par l'apprentissage.

Cette séparation présente un avantage décisif pour la qualité : elle crée des frontières explicites où insérer des contrôles. Un test qui échoue au passage de la couche intermédiaire vers la couche exposée identifie immédiatement le périmètre de la régression.

Le déplacement de la logique de transformation depuis Python vers SQL déclaratif, sous la responsabilité de dbt, constitue un choix architectural fort de ce projet. Il rend la transformation lisible par un analyste, testable de manière déclarative, documentée automatiquement, et surtout traçable : le graphe de dépendances entre modèles est dérivé du code lui-même plutôt que maintenu manuellement.

2.6. Contrats de données et qualité

Un contrat de données est un ensemble de règles qui définit les caractéristiques que les données doivent respecter. Il précise notamment le schéma des données, les types des variables, les valeurs autorisées, les règles d'unicité, la fréquence de mise à jour et les règles liées au domaine métier. Ce contrat permet de rendre les attentes entre les différents composants du système claires et vérifiables. Au lieu de supposer que les données respectent certaines règles, celles-ci sont définies explicitement et peuvent être contrôlées automatiquement.

Dans ce projet, la qualité des données est analysée selon six dimensions principales :

- Complétude : vérifier que les données nécessaires ne sont pas absentes.
- Validité : vérifier que les valeurs respectent les types et les règles définis.
- Unicité : vérifier qu'il n'existe pas de doublons lorsque l'unicité est attendue.
- Cohérence : vérifier que les données sont compatibles entre elles et respectent les règles du système.
- Exactitude : vérifier que les valeurs correspondent correctement à la réalité ou aux sources disponibles.
- Fraîcheur : vérifier que les données sont suffisamment récentes et mises à jour dans les délais attendus.

Ces différentes dimensions sont vérifiées à l'aide de tests automatisés exécutés lors de la reconstruction de l'entrepôt de données. Cela permet de détecter rapidement les problèmes de qualité avant que les données ne soient utilisées par les modèles ou les autres composants de la plateforme.

2.7. La dérive et la surveillance après déploiement

Un modèle déployé se dégrade avec le temps, non parce que son code change, mais parce que le monde change. Trois formes de dérive sont distinguées. La dérive des covariables désigne un changement de la distribution des variables d'entrée sans modification de la relation qui les lie à la cible. La dérive de concept désigne un changement de cette relation elle-même. La dérive des prédictions désigne un déplacement observable de la distribution des sorties du modèle, qui est souvent le premier signal détectable en l'absence de vérité terrain immédiate.

Ce dernier point est crucial en maintenance prédictive : la vérité terrain n'est connue qu'au moment de la défaillance ou de la dépose, soit potentiellement des mois après la prédiction. Une surveillance fondée uniquement sur l'erreur réelle serait donc structurellement en retard. La surveillance de la distribution des prédictions, mise en œuvre dans ce projet, constitue un substitut imparfait mais immédiatement disponible.

2.8. Synthèse et positionnement

L'état de l'art fait apparaître que le pronostic de RUL sur C-MAPSS est un problème bien balisé, pour lequel des performances de référence existent, et que les méthodes classiques d'apprentissage ensembliste y demeurent compétitives dès lors que l'ingénierie des variables

explicatives capture la tendance temporelle de dégradation. Il fait également apparaître que la difficulté industrielle réelle se situe ailleurs : dans la transformation d'un modèle en système.

Le positionnement de ce projet en découle directement. La contribution visée n'est pas d'améliorer l'état de l'art algorithmique sur C-MAPSS, ce qui supposerait des architectures profondes et des moyens de calcul hors de portée du cadre pédagogique, mais de démontrer qu'une performance honnête et vérifiable peut être obtenue à l'intérieur d'une chaîne entièrement industrialisée, reproductible et surveillée.

CHAPITRE 3

Cadre méthodologique et gestion de projet

3.1. Choix d'un cadre agile

Le projet a été conduit selon un cadre agile inspiré de Scrum, adapté aux contraintes d'un travail universitaire en équipe de sept personnes réparties sur des emplois du temps hétérogènes. Ce choix ne relève pas du formalisme : un projet MLOps se prête particulièrement mal à une planification en cascade, car la faisabilité d'une étape ne se révèle souvent qu'au moment où l'étape précédente produit ses premiers artefacts réels.

Trois sprints ont été définis, correspondant à trois paliers de maturité : fondations et ingestion, transformation et modélisation, intégration finale et industrialisation. Chaque sprint s'est clos par une revue documentée et une rétrospective écrite, consignées dans le dépôt.

3.2. Organisation de l'équipe et répartition des responsabilités

L'équipe, constituée sous le nom AeroReliability Lab, a adopté une organisation basée sur une répartition claire des tâches et des responsabilités entre les différents membres. Afin de faciliter le travail en groupe, chaque membre dispose de sa propre branche Git, d'un fichier de suivi des tâches ainsi que d'un périmètre de dossiers dont il est responsable.

Cette organisation permet à chaque membre de travailler de manière indépendante sur les parties qui lui sont attribuées, sans modifier directement le travail des autres membres. Elle permet également de réduire les risques de conflits et de blocages lorsque plusieurs personnes travaillent simultanément sur le projet.

Le suivi des tâches permet de connaître l'avancement de chaque partie du projet et de mieux répartir le travail entre les membres. Chaque contribution est réalisée dans la branche correspondante, puis proposée pour intégration au projet principal à travers une demande de fusion (Pull Request).

Les demandes de fusion permettent de vérifier les modifications réalisées avant leur intégration. Elles facilitent ainsi la collaboration entre les membres tout en permettant de conserver une organisation claire du code et des fichiers du projet. Cette méthode assure également une meilleure traçabilité des différentes contributions réalisées au cours du développement.

Ainsi, l'organisation adoptée combine autonomie dans le travail individuel et coordination au niveau de l'équipe, tout en facilitant l'intégration progressive des différentes parties de la plateforme.

3.3. Le carnet de produit

Le carnet de produit rassemble dix histoires utilisateur hiérarchisées selon trois niveaux de priorité. Chacune est associée à un propriétaire et surtout à des critères d'acceptation vérifiables, formulés en termes d'artefacts produits plutôt que d'intentions.

ID	Pr.	Histoire utilisateur (forme abrégée)	Critère d'acceptation
PB-01	P0	Ingérer les fichiers C-MAPSS dans DuckDB pour disposer d'un entrepôt local fiable	dlt charge FD001–FD004 ; tables brutes présentes ; volumétries documentées
PB-02	P0	Disposer de contrats et de tests pour bloquer les données invalides avant l'entraînement	dbt run réussit ; dbt test passe ; contrats couvrant les trois couches
PB-03	P0	Obtenir un entraînement sans fuite et des références comparatives	Métriques sauvegardées pour moyenne, cycle seul, Ridge, forêt aléatoire et boosting
PB-04	P0	Tracer les expériences et produire un artefact de service reproductible	Métriques et artefacts journalisés dans MLflow ; modèle écrit sur disque
PB-05	P0	Disposer d'un contrat d'API clair pour tester les prédictions	/health et /predict documentés ; charge utile générée fonctionnelle ; champs inconnus rejetés en 422
PB-06	P0	Lancer toute la pile avec une seule commande	docker compose up --build démarre MLflow, l'entraînement, l'API et Dagster
PB-07	P1	Disposer d'une analyse exploratoire étayant la valeur métier	Carnet d'analyse couvrant moteurs, cycles, capteurs et profils de RUL
PB-08	P1	Rendre l'usage de l'API traçable	Prédictions journalisées ; rapport de dérive généré sur commande
PB-09	P1	Produire les artefacts agiles exigés par le module	Plan de sprint, revue, rétrospective, liste de contrôle de livraison, rapport final
PB-10	P2	Garantir une documentation honnête et reproductible	README, guide d'API, licences de données et rapport final référençant des commandes testées

Table 3_Carnet de produit hiérarchisé et critères d'acceptation.

3.4. Déroulement des sprints

3.4.1. Sprint 1 : Fondations

L'objectif du premier sprint était de rendre le projet collaborativement exploitable et d'ingérer des données réelles. Les livrables produits comprennent la structure du dépôt, les branches et fichiers de tâches individuels, la documentation des sources de données et de leurs licences, la chaîne d'ingestion dlt vers DuckDB, et un premier carnet d'analyse exploratoire.

Revue : le jeu de données et l'architecture cible ont été arrêtés, mais l'intégration sur la branche principale est restée insuffisante.

Rétrospective : l'attribution claire des périmètres a permis à chacun de démarrer immédiatement ; en revanche, la branche principale est demeurée vide trop longtemps.

Action corrective adoptée : la branche d'intégration doit être mise à jour au moins une fois par sprint.

3.4.2. Sprint 2 : DataOps et modélisation

L'objectif du deuxième sprint était de transformer la donnée brute et de construire une référence de modélisation défendable. Les livrables comprennent les modèles dbt de couche intermédiaire et exposée, les contrats de données, le document de lignage, la liste de contrôle qualité, les rapports de modèle de référence et de modèle amélioré, ainsi que les premières ébauches de l'API et des utilitaires MLOps.

Revue : un travail de composant de bonne qualité existait, mais les branches n'étaient pas fusionnées et MLflow n'était pas connecté au service.

Rétrospective : l'indépendance de travail a bien fonctionné ; la connexion entre les

composants a été le point faible.

Action corrective adoptée : chaque branche doit disposer d'un contrat d'intégration testé avant la semaine finale.

3.4.3. Sprint 3 : Intégration finale

L'objectif du troisième sprint était de transformer un ensemble de composants épars en une plateforme unique et reproductible. Les travaux ont porté sur la fusion des branches dans [integration/final](#) , l'entraînement complet sur les quatre sous-ensembles avec production des preuves, le durcissement du schéma d'API et la suppression du repli silencieux, la mise en place de la pile Docker Compose en une commande, et l'établissement des barrières d'intégration continue.

Revue : la branche finale constitue la version candidate à la livraison.

Rétrospective : l'intégration a rendu le projet démontrable à partir d'une seule branche, mais des livrables manquants de deux membres ont dû être reconstruits pendant la phase d'intégration.

Action corrective pour l'avenir : créer les tickets et jalons dès le début et imposer une revue par les pairs avant toute fusion.

3.5. Définition de « terminé »

Une exigence a été formalisée pour qualifier un travail d'achevé, afin d'éviter le phénomène classique du « c'est presque fini » qui masque une dette d'intégration :

1. Le code est validé sur la branche d'intégration.
2. Des tests ou une trace d'exécution existent pour la zone modifiée.
3. Les données brutes générées et les bases locales d'exécution ne sont pas versionnées.
4. La documentation nomme des commandes exactes, leurs sorties attendues et leurs limites.
5. Le comportement de démonstration utilise le modèle réellement entraîné, sauf activation explicite du repli dans un test.

3.6. Analyse critique de la conduite de projet

Une lecture honnête du déroulement fait apparaître un écart classique entre l'organisation nominale et l'organisation effective. Le modèle « une branche par personne » a atteint son objectif d'autonomie mais a révélé son coût : en l'absence de fusions régulières, l'intégration a été reportée en fin de projet et concentrée sur un petit nombre de contributeurs. Deux branches n'ont pas produit de livrable exploitable, contraignant l'intégration à reconstruire les documents d'API, les charges utiles de démonstration et les artefacts agiles.

Cet enseignement dépasse le cadre du projet. Il illustre le principe selon lequel l'intégration continue est d'abord une discipline sociale avant d'être un outil technique : une chaîne d'intégration automatisée ne produit de valeur que si les contributions y sont soumises fréquemment. La leçon retenue, consignée dans la rétrospective finale, est qu'un contrat d'interface testé doit exister entre modules dès le premier sprint, même sous une forme minimale, afin que chaque contributeur puisse valider son travail contre une cible stable.

CHAPITRE 4

Le jeu de données NASA C-MAPSS et son analyse

4.1. Origine et nature des données

Le jeu de données utilisé est le Turbofan Engine Degradation Simulation Data Set produit par le NASA Ames Prognostics Center of Excellence. Il a été généré à l'aide du simulateur C-MAPSS (Commercial Modular Aero-Propulsion System Simulation), un modèle haute-fidélité de turboréacteur commercial permettant d'injecter des dégradations progressives dans différents modules de la machine et d'observer la réponse des capteurs embarqués.

Ce jeu de données s'est imposé comme la référence de facto du pronostic de durée de vie résiduelle, pour trois raisons : il est public et librement redistribuable pour la recherche, il fournit des trajectoires complètes jusqu'à défaillance, et il propose un protocole d'évaluation standardisé permettant la comparaison entre publications.

Sa nature simulée constitue simultanément sa force et sa limite principale. Force, parce qu'elle permet de disposer d'une vérité terrain parfaite sur l'instant de défaillance, ce qui serait impossible sur des données industrielles réelles où les données sont souvent préventives. Limite, parce que le bruit et les modes de défaillance y sont générés par un modèle et ne reproduisent pas toute la complexité d'une flotte réelle : absence de valeurs manquantes accidentelles, de pannes de capteurs, de changements de calibration ou d'interventions de maintenance intermédiaires.

4.2. Structure des fichiers

Le jeu de données se compose de quatre sous-ensembles indépendants, désignés FD001 à FD004, chacun décliné en trois fichiers : un fichier d'entraînement, un fichier de test et un fichier d'étiquettes de RUL pour le test.

Sous-ensemble	Conditions opérationnelles	Modes de défaillance	Difficulté relative
FD001	Une seule condition (niveau de la mer)	Un seul mode : dégradation du compresseur haute pression	La plus faible
FD002	Six conditions opérationnelles distinctes	Un seul mode	Moyenne variabilité des régimes
FD003	Une seule condition	Deux modes : compresseur et ventilateur	Moyenne ambiguïté du mode
FD004	Six conditions opérationnelles	Deux modes	La plus élevée

Table 4— Caractéristiques des quatre sous-ensembles C-MAPSS.

Chaque ligne d'un fichier décrit un cycle de fonctionnement d'une unité et comporte vingt-six colonnes : l'identifiant de l'unité, le numéro de cycle, trois réglages opérationnels et vingt-et-une mesures de capteurs. Les capteurs correspondent à des grandeurs physiques telles que des températures et pressions en différents points du flux, des vitesses de rotation des arbres, des débits de carburant et des ratios de purge.

Un choix méthodologique important a été fait dans ce projet : **les quatre sous-ensembles sont traités simultanément** dans un unique entraînement, avec un identifiant de sous-ensemble ajouté comme variable explicative. Une part importante de la littérature se limite au seul FD001, nettement plus facile. L'entraînement conjoint expose le modèle à une variabilité de conditions opérationnelles et de modes de

défaillance beaucoup plus large, ce qui rend les résultats plus difficiles à obtenir mais plus représentatifs d'un contexte de flotte hétérogène.

4.3. Volumétrie effective

L'exécution finale de la chaîne, dont les traces sont consignées dans le fichier [reports/model_metrics/final_evaluation.json](#), a porté sur les volumétries suivantes.

Grandeur	Valeur	Commentaire
Lignes brutes d'entraînement	160 359	Ensemble des cycles observés des unités d'entraînement, FD001 à FD004
Lignes brutes de test	104 897	Trajectoires tronquées avant défaillance
Moteurs d'entraînement	709	Chaque trajectoire va jusqu'à la défaillance
Moteurs de test	707	Une étiquette de RUL par moteur au dernier cycle
Lignes d'entraînement après amorçage des fenêtres	157 523	Perte de 2 836 lignes liée aux quatre premiers cycles de chaque unité
Lignes de test en ligne après amorçage	102 069	Même mécanisme d'amorçage
Lignes dans la table exposée dbt	265 256	Entraînement et test réunis dans la table de variables explicatives
Nombre de variables explicatives finales	89	Voir chapitre 8

Table 5— Volumétrie effective de l'exécution finale.

La perte de lignes liée à l'amorçage des fenêtres glissantes mérite un commentaire. Les statistiques calculées sur une fenêtre de cinq cycles ne sont pleinement définies qu'à partir du cinquième cycle de chaque trajectoire. Les quatre premières lignes de chaque unité sont donc écartées de l'entraînement et de l'évaluation. Avec 709 unités d'entraînement, cela représente exactement 2 836 lignes, ce qui correspond à l'écart observé et confirme la cohérence interne de la chaîne. Cette contrainte a une conséquence opérationnelle directe, documentée dans l'API : la rediffusion d'un moteur exige au minimum cinq cycles observés.

4.4. Analyse exploratoire

4.4.1. Distribution des durées de vie

L'analyse exploratoire conduite sur les trajectoires d'entraînement fait apparaître une dispersion considérable des durées de vie entre unités. Certaines défont après un peu plus de cent vingt cycles, d'autres dépassent trois cent cinquante cycles. Cette dispersion est précisément ce qui justifie la maintenance prédictive : une politique de remplacement systématique calibrée sur la plus faible durée de vie gaspillerait plus des deux tiers de la vie utile des unités les plus robustes, tandis qu'une politique calibrée sur la moyenne provoquerait des défaillances en service pour une part significative de la flotte.

4.4.2. Comportement des capteurs

Les vingt-et-un canaux de capteurs ne se comportent pas de manière homogène. Trois familles se dégagent.

- Des capteurs quasi constants sur toute la trajectoire, dont la variance est nulle ou négligeable dans les sous-ensembles à condition opératoire unique. Ils n'apportent aucune information de dégradation dans ce contexte, mais deviennent informatifs dans les sous-ensembles multi-conditions où ils encodent le régime de fonctionnement.
- Des capteurs monotones croissants ou décroissants, dont la valeur dérive progressivement à mesure que la dégradation s'installe. Ce sont les porteurs principaux du signal de pronostic.
- Des capteurs bruités dont la tendance existe mais est noyée dans une variabilité cycle à cycle importante ; leur exploitation exige un lissage, ce qui motive directement les variables de moyenne glissante.

Cette observation est structurante pour l'ingénierie des variables explicatives : la valeur instantanée d'un capteur est un indicateur faible, alors que sa tendance sur quelques cycles est un indicateur beaucoup plus robuste.

4.4.3. Profil temporel de la dégradation

Le profil de dégradation observé est typiquement non linéaire : une phase initiale longue durant laquelle les capteurs restent proches de leur valeur nominale, suivie d'une phase d'accélération où la dérive devient nette et rapide. Cette forme justifie doublement l'écrtage de la RUL : elle confirme qu'aucun signal exploitable n'existe en début de vie, et elle indique que l'utilité opérationnelle du modèle se concentre sur la phase terminale, celle où la décision de maintenance est effectivement prise.

4.4.4. Effet des conditions opérationnelles

Dans les sous-ensembles FD002 et FD004, les trois réglages opérationnels définissent six régimes discrets. La lecture brute des capteurs y est dominée par le régime plutôt que par la dégradation : deux cycles de même niveau de santé mais de régimes différents produisent des valeurs de capteurs très éloignées. C'est la raison principale pour laquelle ces sous-ensembles sont réputés plus difficiles, et la raison pour laquelle les réglages opérationnels sont explicitement inclus dans le vecteur de variables explicatives : ils permettent au modèle de conditionner son interprétation des capteurs au régime observé.

4.5. Licence et gouvernance des données

Le jeu de données appartient au domaine public américain et est distribué par la NASA à des fins de recherche. Le dépôt du projet respecte trois règles de gouvernance explicites.

- Les fichiers de données ne sont jamais versionnés dans le dépôt Git, afin d'éviter l'alourdissement de l'historique et toute ambiguïté sur la propriété du contenu.
- Le téléchargement est automatisé par le script [scripts/download_data.py](#), ce qui garantit que tout évaluateur obtient exactement la même source.
- La source, la citation et les conditions d'usage sont documentées dans [docs/data-sources-and-licenses.md](#).

Le code et la documentation du projet sont pour leur part publiés sous licence MIT. Cette séparation entre licence du code et licence des données est une bonne pratique de

gouvernance souvent négligée dans les projets étudiants.

CHAPITRE 5

Architecture de la plateforme

5.1. Principes directeurs

Quatre principes ont guidé la conception de l'architecture.

- **Séparation stricte des couches** : la donnée brute, la donnée nettoyée et la donnée exposée résident dans des schémas distincts, jamais écrasés les uns par les autres. Toute reconstruction repart de la couche brute.
- **Contrats explicites entre modules** : chaque interface entre deux responsabilités ingestion et transformation, transformation et apprentissage, apprentissage et service est formalisée par un document et vérifiée par un test.
- **Reproductibilité par défaut** : toute étape doit être rejouable par une commande unique sur un poste vierge, sans intervention manuelle non documentée.
- **Refus de la dissimulation** : le système préfère échouer visiblement plutôt que de produire un résultat plausible mais faux. Ce principe a une traduction concrète dans le service de prédiction, qui refuse de démarrer sans modèle valide.

5.2. Vue d'ensemble du flux

Le flux de bout en bout suit une progression linéaire de la source vers la décision, chaque étape produisant un artefact persistant et vérifiable.

```
Fichiers bruts de capteurs (NASA C-MAPSS)
|
v
Ingestion dlt
|
v
DuckDB — schémas raw et staging
|
v
Transformations et tests dbt
|
v
Table de variables explicatives (marts)
|
v
Chaîne d'entraînement scikit-learn
|
v
MLflow — suivi et registre de modèles
|
v
Service de prédiction FastAPI
|
v
Docker, GitHub Actions, surveillance, démonstration
```

Figure 1— Chaîne de traitement de bout en bout de la plateforme.

5.3. Justification des choix technologiques

Chaque brique a été retenue pour des raisons explicites, arbitrées entre puissance fonctionnelle et coût de mise en œuvre dans un contexte local.

Brique	Outil retenu	Justification du choix	Alternative écartée
Ingestion	dlt	Bibliothèque Python déclarative, schéma inféré, chargement idempotent, aucune infrastructure requise	Scripts pandas ad hoc non idempotents, sans gestion de schéma
Entrepôt	DuckDB	Moteur analytique en processus, aucune administration, performances colonnaires élevées sur un poste unique	PostgreSQL coût opérationnel injustifié en local
Transformation	dbt	SQL déclaratif versionné, tests natifs, lignage dérivé du code, documentation automatique	Transformations Python logique opaque et non testable déclarativement
Orchestration	Dagster	Modèle orienté actifs, adapté aux dépendances de données, interface de matérialisation	Airflow modèle orienté tâches, plus lourd à installer
Modélisation	scikit-learn	Écosystème mature, implémentations éprouvées, sérialisation simple, exigence du module	PyTorch surdimensionné pour des données tabulaires
Suivi et registre	MLflow	Standard de fait, registre avec alias de promotion, intégration native scikit-learn	Suivi manuel par fichiers non traçable
Service	FastAPI	Validation par Pydantic, documentation OpenAPI automatique, performances asynchrones	Flask validation manuelle, documentation à maintenir
Conteneurisation	Docker Compose	Pile complète en une commande, autorisée explicitement par l'encadrement	Kubernetes hors périmètre pédagogique
Intégration continue	GitHub Actions	Intégration native au dépôt, exécution de la chaîne complète	Exécution locale seule non vérifiable par un tiers

Table 6— Choix technologiques et justifications.

5.4. Modèle en couches de l'entrepôt

L'entrepôt DuckDB est organisé en trois schémas dont les responsabilités sont disjointes.

Schéma	Table	Produite par	Rôle
raw	raw_sensor_readings	dlt	Copie fidèle des mesures brutes de capteurs
raw	raw_rul_labels	dlt	Étiquettes de RUL des trajectoires de test
staging	stg_sensor_readings	dbt	Lectures nettoyées, typées et filtrées
staging	stg_rul_labels	dbt	Étiquettes nettoyées, unicité garantie par moteur
marts	fct_equipment_health_features	dbt	Table de variables explicatives prête pour l'apprentissage

Table 7— Organisation en schémas de l'entrepôt DuckDB.

Cette structure garantit qu'une erreur de logique métier détectée tardivement peut être corrigée en modifiant un modèle dbt et en relançant la transformation, sans jamais devoir retélécharger ni réingérer la source. C'est l'un des bénéfices opérationnels les plus tangibles de l'approche ELT par rapport à l'approche ETL classique.

5.5. Architecture de service

La pile d'exécution comprend cinq services orchestrés par Docker Compose.

Service	Rôle	Port exposé
mlflow	Serveur de suivi et registre de modèles, adossé à SQLite et à un dossier d'artefacts	5000
training-init	Service d'initialisation : téléchargement des données puis exécution du travail Dagster complet	--
api	Service FastAPI de prédiction et console d'ingénierie	8000
dagster-webserver	Interface de matérialisation et de supervision des actifs	3000
dagster-daemon	Démon de planification pour l'exécution programmée quotidienne	--

Table 8— Services de la pile d'exécution.

Le service `training-init` mérite une attention particulière : il matérialise le principe de reproductibilité. Au premier démarrage, il télécharge les données, exécute l'ingestion, la transformation, les tests, l'entraînement réel dans un carnet exécuté, l'enregistrement dans MLflow et la génération des preuves de surveillance. Le service d'API ne démarre qu'après, et uniquement si un modèle valide est disponible. L'évaluateur obtient donc, à partir d'une seule commande, non pas une démonstration préfabriquée mais une reconstruction complète.

5.6. Une console d'ingénierie comme surface de démonstration

Une interface web a été développée au-dessus du service. Il importe de préciser sa nature exacte, car elle a été une source d'ambiguïté potentielle : il ne s'agit pas d'un affichage d'aéronefs en vol réel, mais d'une console d'ingénierie qui rejoue les trajectoires de test C-MAPSS stockées dans DuckDB et effectue une véritable ingénierie de variables et une véritable inférence côté serveur.

Cinq vues la composent : le triage de flotte, la rediffusion cycle par cycle d'un moteur, la prédiction unitaire ou par lot, les preuves de plateforme, et la documentation du projet. Deux modes de fonctionnement coexistent : un mode opérations, qui masque délibérément la vérité terrain future pour reproduire les conditions réelles d'exploitation, et un mode évaluation, qui expose la RUL réelle et l'erreur de prédiction à des fins de validation académique.

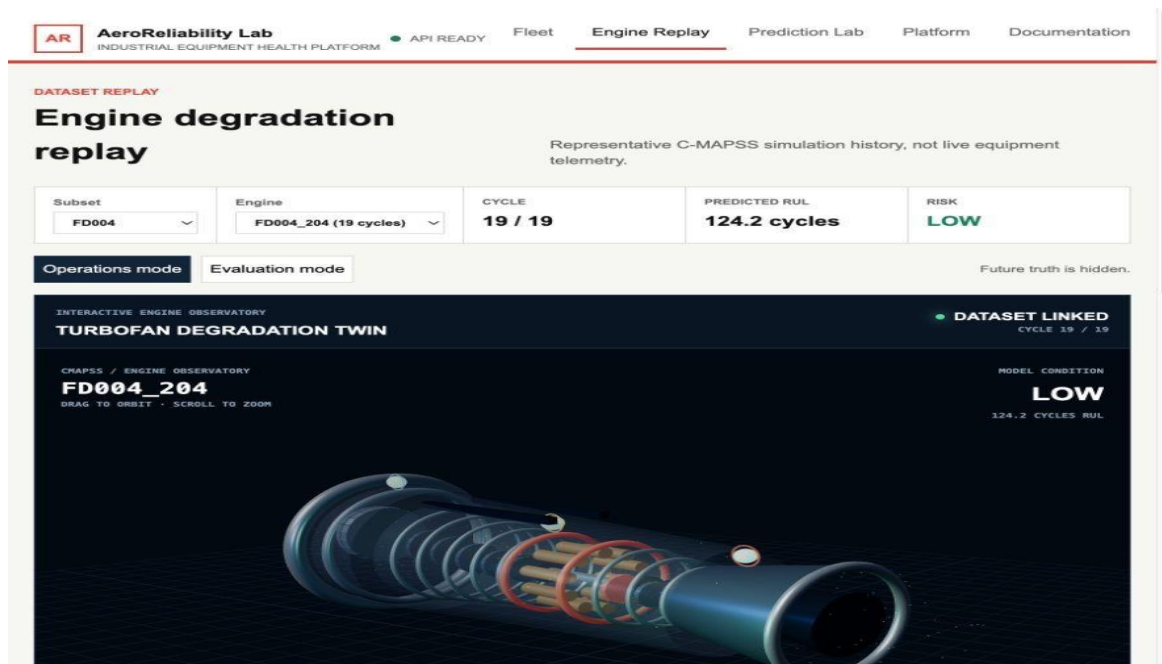


Figure 2 — Vue de la console d'ingénierie : observatoire de dégradation d'un turboréacteur.

La visualisation tridimensionnelle du turboréacteur qu'elle intègre associe les composants ventilateur, compresseur, chambre de combustion, turbine, échappement à l'état de dégradation du cycle sélectionné. Cette association constitue une cartographie visuelle explicative, et non une simulation physique validée. Cette précision est explicitement portée dans la documentation du dépôt, conformément au principe de refus de la dissimulation.

CHAPITRE 6

La chaîne DataOps : ingestion, stockage, transformation, orchestration

6.1. L'ingestion avec dlt

L'ingestion constitue la frontière entre le monde extérieur, non maîtrisé, et l'entrepôt, maîtrisé. Sa qualité conditionne tout ce qui suit. Trois exigences ont été posées : l'ingestion doit être reproductible, idempotente et traçable.

6.1.1. Implémentation

La bibliothèque dlt a été retenue pour son modèle déclaratif. Le développeur définit des ressources sous forme de générateurs Python et une source qui les regroupe ; dlt se charge de l'inférence de schéma, de la normalisation des noms de colonnes, de la gestion des paquets de chargement et de l'écriture dans la destination DuckDB. Le module d'ingestion du projet définit deux ressources principales : `sensor_readings` pour les mesures et `rul_labels` pour les étiquettes de test.

Chaque fichier brut C-MAPSS est un fichier texte à séparateur espace sans en-tête. Le chargement reconstruit donc explicitement les noms de colonnes et conserve le nom du fichier source dans une colonne `source_file`. Ce dernier point, apparemment anodin, est en réalité déterminant : c'est cette colonne qui permet, en aval, de reconstituer à la fois l'appartenance au sous-ensemble et la partition entraînement/test, sans avoir à maintenir une table de correspondance externe.

6.1.2. Traçabilité du chargement

Chaque exécution produit un paquet de chargement horodaté dont l'état est vérifiable. La sortie attendue de l'étape d'ingestion, documentée dans la chaîne locale, indique explicitement que le paquet est chargé et ne contient aucun travail en échec. Ce contrôle constitue la première barrière de qualité de la chaîne.

6.2. DuckDB comme entrepôt analytique local

DuckDB est un système de gestion de base de données analytique fonctionnant en processus, sans serveur ni configuration. Son modèle de stockage colonnaire et son moteur vectorisé lui confèrent des performances comparables à celles d'entrepôts distribués sur des volumes tenant sur une machine, ce qui est le cas ici avec environ 265 000 lignes.

Le choix de DuckDB répond directement à la contrainte pédagogique de déploiement local. Un entrepôt client-serveur classique aurait introduit une dépendance d'infrastructure, une gestion d'identifiants et un point de défaillance supplémentaire, sans bénéfice pour un volume de cette taille. La base réside dans un fichier unique, `cmapss_ingestion.duckdb`, exclu du versionnement Git et monté comme volume dans la pile Docker.

Choix assumé : le volume de données du projet ne justifie aucune infrastructure distribuée. Recourir à Spark ou à un entrepôt cloud aurait constitué une complexité ostentatoire sans contrepartie mesurable, contraire au principe de sobriété architecturale retenu.

6.3. Les transformations dbt

6.3.1. Le modèle de couche intermédiaire

Le modèle `stg_sensor_readings` réalise le nettoyage technique. Il sélectionne explicitement les colonnes attendues identifiant du moteur, cycle, trois réglages, vingt-et-un capteurs, fichier source et filtre les lignes dont l'identifiant ou le cycle est nul. La sélection explicite plutôt qu'un `select *` est un choix délibéré : elle transforme toute modification de schéma en amont en erreur visible plutôt qu'en propagation silencieuse.

```
select
    engine_id, cycle,
    setting_1, setting_2, setting_3,
    sensor_1, ..., sensor_21,
    source_file
from {{ source('raw', 'raw_sensor_readings') }}
where engine_id is not null
    and cycle is not null
```

6.3.2. Le modèle de couche exposée

Le modèle `fct_equipment_health_features` constitue le cœur de la chaîne DataOps. Il construit la table de variables explicatives consommée par l'entraînement, et intègre plusieurs transformations à forte valeur ajoutée.

- **Dérivation de la partition** : la colonne `split` est calculée à partir du préfixe du nom de fichier source, distinguant entraînement et test sans table externe.
- **Extraction de l'identifiant de sous-ensemble** : une expression régulière extrait le numéro FD00x du nom de fichier et le convertit en entier, fournissant au modèle une variable de contexte.
- **Transformation causale du cycle** : la variable `cycle_log1p` applique un logarithme au cycle courant. Elle ne dépend que du présent et ne présuppose aucune connaissance de la durée de vie totale.
- **Discrétisation de l'âge** : la variable `engine_age_bucket` classe le cycle courant en trois catégories : précoce, intermédiaire, tardif selon des seuils fixes de 75 et 150 cycles.
- **Statistiques glissantes** : des moyennes et écarts-types sur cinq cycles sont calculés pour une sélection de capteurs jugés informatifs, au moyen de fonctions de fenêtrage SQL.

6.3.3. La causalité des fenêtres glissantes

Le point le plus critique de ce modèle réside dans la définition exacte des fenêtres. Toutes les fonctions de fenêtrage utilisent la clause suivante :

```
over (partition by engine_id order by cycle rows between 4 preceding and current row)
```

Trois éléments de cette clause sont indispensables et leur omission constituerait une faute méthodologique caractérisée.

- La partition par `engine_id` garantit qu'aucune fenêtre ne chevauche deux moteurs distincts. Sans elle, la moyenne glissante du premier cycle d'un moteur inclurait les derniers cycles du moteur précédent, ce qui mélangerait des trajectoires indépendantes.
- L'ordonnancement par `cycle` garantit que la fenêtre suit bien l'ordre temporel réel et non l'ordre physique de stockage.

- La borne `between 4 preceding and current row` garantit la **causalité** : la fenêtre ne regarde que le passé et le présent. Une fenêtre centrée, qui inclurait des cycles futurs, injecterait dans la variable explicative une information indisponible au moment de la prédiction réelle une fuite d'information temporelle.

Le commentaire d'en-tête du modèle dbt documente explicitement cette propriété : chaque valeur ne dépend que du cycle courant ou des cycles précédents, et aucun cycle final de moteur n'est utilisé, ce qui évite toute fuite entre l'entraînement et le service.

6.4. Orchestration avec Dagster

Dagster adopte un modèle orienté **actifs** plutôt qu'orienté tâches. La différence est conceptuellement importante : on ne déclare pas « exécuter le script d'ingestion » mais « l'actif `raw_sensor_data` doit exister et être à jour ». Cette orientation correspond mieux à la nature d'une chaîne de données, où l'objet d'intérêt est l'état des tables plutôt que l'ordonnancement des commandes.

Actif	Description	Dépend de
<code>raw_sensor_data</code>	Exécute l'ingestion dlt vers le schéma brut de DuckDB	—
<code>dbt_transform</code>	Exécute dbt run pour les modèles de couches intermédiaire et exposée	<code>raw_sensor_data</code>
<code>dbt_test</code>	Exécute dbt test, barrière de qualité bloquante	<code>dbt_transform</code>
<code>feature_table_validation</code>	Valide le nombre de lignes de <code>stg_sensor_readings</code>	<code>dbt_test</code>
<code>feature_engineering</code>	Valide le nombre de lignes de <code>fct_equipment_health_features</code>	<code>dbt_test</code>
<code>model_training</code>	Exécute le carnet d'entraînement réel, évalue et enregistre dans MLflow	<code>feature_engineering</code>
<code>drift_report</code>	Produit les preuves de surveillance après enregistrement du modèle	<code>model_training</code>

Table 9— Les sept actifs du graphe d'orchestration Dagster.

Les sept actifs forment un graphe de dépendances unique, planifié quotidiennement à six heures. La matérialisation effectuée au démarrage de la pile Docker est conservée dans le stockage partagé de Dagster, de sorte que l'état du graphe survit au redémarrage des conteneurs.

Le positionnement de `dbt_test` en amont de tout entraînement constitue le choix d'architecture le plus significatif de cette chaîne : **il est structurellement impossible d'entraîner un modèle sur des données ayant échoué aux tests de qualité**. La qualité n'est pas une vérification a posteriori mais une condition d'exécution.

6.5. Exécution locale et vérification

Une voie d'exécution sans interface est fournie pour la vérification rapide et pour l'intégration continue :

```
PYTHONPATH=src python orchestration/run_local.py
```

Cette commande enchaîne trois étapes dont les sorties attendues sont documentées : le chargement dlt, la construction de trois modèles dbt, et l'exécution de huit tests de qualité. La

chaîne est complétée par onze tests Python portant sur la couche DataOps, exécutés par pytest.

Étape	Résultat attendu	Statut vérifié
Ingestion dlt	Paquet chargé, aucun travail en échec, FD001–FD004 complets	Vérifié
dbt run	3 modèles construits, 0 avertissement, 0 erreur	Vérifié
dbt test	8 tests réussis sur 8	Vérifié
pytest DataOps	11 tests réussis	Vérifié

Table 10— Résultats de l'exécution locale de la chaîne DataOps.

CHAPITRE 7

Gouvernance de la qualité : contrats, tests et lignage

7.1. Pourquoi une gouvernance explicite

Dans un projet mené par sept personnes, la donnée circule entre des responsabilités distinctes : celui qui ingère n'est pas celui qui transforme, qui n'est pas celui qui entraîne. Chaque frontière est un lieu d'ambiguïté potentielle. Un producteur peut renommer une colonne en toute bonne foi ; un consommateur découvre la rupture plusieurs jours plus tard, sous forme d'un modèle dont les performances s'effondrent sans raison apparente.

La réponse apportée est de transformer chaque attente implicite en engagement explicite et exécutable. Trois mécanismes complémentaires y concourent : les contrats de données, qui documentent l'engagement

les tests dbt, qui l'appliquent automatiquement ; et la documentation du lignage, qui rend visible la propagation d'une modification.

7.2. Les contrats de données

Trois contrats ont été rédigés, un par couche de l'entrepôt, chacun avec un propriétaire nommé et un mécanisme d'application identifié.

Contrat	Couche	Mécanisme d'application
DC-001 raw_sensor_readings	Brute	Contrôles d'ingestion et schéma dbt
DC-002 stg_sensor_readings	Intermédiaire	Tests déclaratifs dbt
DC-003 fct_equipment_health_features	Exposée	Tests dbt et tests Python de la couche DataOps

Table 11— Contrats de données et responsabilités associées.

Chaque contrat spécifie le schéma attendu, les règles de qualité applicables, les critères d'acceptation et le lignage amont et aval. Le contrat de la couche exposée joue un rôle particulier : il constitue l'interface formelle entre l'équipe données et l'équipe modélisation. Il précise le chemin du fichier de base, le schéma, le nom de la table, la liste exhaustive des colonnes et la volumétrie attendue de 265 256 lignes. L'équipe de modélisation peut ainsi développer contre une cible stable sans dépendre d'échanges informels.

7.3. Les six dimensions de la qualité

7.3.1. Complétude

La complétude exige que les champs requis contiennent effectivement une valeur. Elle est vérifiée par des tests de non-nullité appliqués aux colonnes structurantes de chaque couche : identifiant du moteur et cycle en couche intermédiaire, et l'ensemble des colonnes dérivées en couche exposée. L'objectif retenu est de cent pour cent pour la couche exposée, car l'entraînement d'un modèle sur des variables partiellement nulles produirait un

comportement imprévisible au service.

7.3.2. Validité

La validité porte sur la conformité des valeurs à leur domaine attendu : types de données, plages numériques, absence de valeurs non finies, et appartenance à une liste fermée pour les variables catégorielles. Le test le plus caractéristique est celui appliqué à `engine_age_bucket`, qui n'admet que trois valeurs : précoce, intermédiaire et tardif. Toute valeur inattendue ferait échouer la construction.

7.3.3. Unicité

L'unicité garantit l'absence de doublons. Elle est appliquée strictement aux étiquettes de RUL, où un moteur ne doit apparaître qu'une seule fois. Son enjeu dépasse la propreté formelle : un doublon dans les étiquettes fausserait le poids relatif d'un moteur dans l'évaluation, et un doublon dans les mesures perturberait les fenêtres glissantes en dupliquant un cycle dans le calcul de la tendance.

7.3.4. Cohérence

La cohérence porte sur la logique interne des données. La règle la plus importante dans ce contexte est la **monotonie décroissante de la RUL** : pour un moteur donné, la durée de vie résiduelle ne peut qu'être décroissante à mesure que le cycle augmente. Toute violation de cette règle signalerait une erreur de calcul de la cible ou un mélange de trajectoires.

7.3.5. Exactitude

L'exactitude désigne l'adéquation entre la donnée et la réalité qu'elle représente. Elle est ici largement acquise par nature, le jeu de données étant produit par un simulateur de référence et non par une chaîne d'acquisition faillible. Le contrôle résiduel consiste en une détection de valeurs aberrantes en couche intermédiaire.

7.3.6. Fraîcheur

La fraîcheur est une dimension à portée limitée dans le contexte de ce projet, les données étant historiques et chargées une fois. Elle est néanmoins documentée avec des engagements de service, car elle constitue une dimension structurante d'un déploiement réel où l'entrepôt serait alimenté en continu.

7.4. Les tests dbt effectivement implémentés

La table suivante recense les huit tests exécutés à chaque reconstruction de l'entrepôt. Leur nombre est modeste, ce qui appelle une remarque de transparence : ces tests couvrent les contraintes structurantes et sont tous effectivement exécutés et vérifiés, alors que la stratégie de qualité documentée envisage des contrôles plus étendus plages de valeurs par capteur, décroissance de la RUL, contrôles de volumétrie qui relèvent de la feuille de route et non de l'implémentation actuelle.

Modèle	Colonne	Test appliqué
stg_sensor_readings	engine_id	not_null
stg_sensor_readings	cycle	not_null
stg_rul_labels	engine_id	not_null et unique
fct_equipment_health_features	engine_id	not_null

fct_equipment_health_features	cycle_log1p	not_null
fct_equipment_health_features	engine_age_bucket	not_null et accepted_values (early / middle / late)
fct_equipment_health_features	split	not_null et accepted_values (train / test)
fct_equipment_health_features	subset_id	not_null

Table 12— Tests de qualité dbt exécutés à chaque reconstruction (8 tests, 8 réussis).

Cette distinction entre l'implémenté et le projeté est délibérément maintenue dans ce rapport. Présenter une stratégie de qualité ambitieuse comme si elle était intégralement déployée constituerait exactement le type de dissimulation que le projet s'est fixé de proscrire.

7.5. Le lignage des données

Le lignage documente le trajet complet de la donnée, de la source externe jusqu'aux prédictions. Sa valeur est double : il permet l'analyse d'impact quelles tables et quels modèles sont affectés si une colonne source change et il permet l'analyse de cause racine d'où provient une valeur anormale observée en aval.

Le lignage effectif du projet suit la séquence suivante :

```
Fichiers NASA C-MAPSS (train / test / RUL, FD001..FD004)
-> raw.raw_sensor_readings      [dlt]
-> raw.raw_rul_labels           [dlt]
    -> staging.stg_sensor_readings [dbt]
    -> staging.stg_rul_labels      [dbt]
        -> marts.fct_equipment_health_features [dbt]
            -> jeu d'entraînement scikit-learn
                -> modèle enregistré MLflow
                    -> logs/prediction logs.jsonl
```

Figure 3— Lignage effectif de la donnée brute jusqu'aux journaux de prédiction.

Un point d'attention méthodologique doit être signalé. La documentation de lignage produite lors du sprint 2 décrit une architecture plus riche que celle finalement implémentée : elle mentionne une couche intermédiaire distincte comportant trois modèles fenêtres de cycle de vie, taux de dégradation et statistiques par moteur ainsi que des volumétries correspondant au seul sous-ensemble FD001. L'implémentation finale a consolidé ces transformations dans un unique modèle de couche exposée et a étendu le périmètre aux quatre sous-ensembles.

Cet écart entre document de conception et implémentation est un phénomène banal en ingénierie, mais il illustre un risque réel de la gouvernance documentaire : une documentation qui n'est pas régénérée à partir du code diverge inévitablement de celui-ci. La leçon retenue, inscrite au chapitre des perspectives, est de substituer à la documentation manuelle de lignage la documentation générée automatiquement par dbt, qui dérive le graphe des dépendances du code lui-même et ne peut donc pas diverger.

7.6. La stratégie de prévention des fuites d'information

Une fuite d'information désigne la présence, dans les variables explicatives d'entraînement, d'une information qui ne sera pas disponible au moment de la prédiction réelle. Elle produit un effet particulièrement pernicieux : des performances excellentes en

évaluation et un effondrement en production. Quatre risques ont été identifiés et traités explicitement.

7.6.1. Partition par ligne au lieu de partition par unité

Une validation croisée découpant les lignes aléatoirement placerait des cycles d'un même moteur simultanément dans l'ensemble d'apprentissage et dans l'ensemble de validation. Le modèle mémoriserait alors la trajectoire propre de ce moteur plutôt que d'apprendre un mécanisme généralisable. La solution mise en œuvre est une validation croisée **par groupes**, où le groupe est l'identifiant du moteur : aucune unité n'apparaît dans deux plis différents.

7.6.2. Variables construites à partir du futur

Une moyenne glissante centrée ou décalée vers l'avenir intégrerait des cycles postérieurs au point de prédiction. La solution, décrite au chapitre précédent, consiste à imposer des fenêtres strictement causales, à la fois dans le modèle dbt et dans la chaîne d'entraînement Python.

7.6.3. Ajustement des normalisateurs sur l'ensemble des données

Ajuster un normalisateur sur la concaténation des données d'entraînement et de test transmettrait au modèle une information sur la distribution du test. La solution consiste à encapsuler la normalisation dans une chaîne scikit-learn, ajustée exclusivement sur l'entraînement puis appliquée au test.

7.6.4. Choix d'hyperparamètres guidé par le test

Le seuil d'écrtage de la RUL aurait pu être ajusté en observant la performance de test. Il a au contraire été fixé à 125 cycles a priori, sur la base de la distribution d'entraînement et de la pratique établie dans la littérature, avant toute évaluation.

CHAPITRE 8

Ingénierie des variables explicatives et modélisation

8.1. Formulation du problème d'apprentissage

Le problème est formulé comme une régression supervisée : à partir d'un vecteur de variables explicatives décrivant l'état d'un moteur à un cycle donné, prédire une valeur continue représentant la durée de vie résiduelle écrêtée à 125 cycles.

Une alternative aurait consisté à formuler le problème en classification, par exemple en prédisant directement une classe de risque. Ce choix a été écarté pour deux raisons. D'une part, la régression conserve l'information ordinale complète et permet de dériver a posteriori n'importe quelle politique de seuils, alors que la classification fige la politique dans le modèle. D'autre part, l'information de RUL en cycles est directement actionnable par un planificateur de maintenance, ce qu'une classe abstraite n'est pas.

8.2. Construction des variables explicatives

Le vecteur final compte **89 variables**, organisées en quatre familles.

Famille	Nombre	Description	Justification
Contexte	2	subset_id, cycle	Identifie le régime de conditions et le mode de défaillance ; le cycle situe la position dans la trajectoire
Réglages opérationnels	3	op_setting_1 à op_setting_3	Permet au modèle de conditionner l'interprétation des capteurs au régime de fonctionnement
Capteurs bruts	21	sensor_1 à sensor_21	Valeur instantanée de chaque canal de mesure
Statistiques glissantes	63	Moyenne, écart-type et pente sur 5 cycles pour chacun des 21 capteurs	Capture la tendance de dégradation, signal beaucoup plus robuste que la valeur instantanée

Table 13— Composition du vecteur de 89 variables explicatives.

8.2.1. La pente glissante

La variable la plus discriminante de la troisième famille est la pente sur cinq cycles, obtenue par régression linéaire locale de la valeur du capteur sur le numéro de cycle à l'intérieur de la fenêtre. Sa supériorité conceptuelle sur la moyenne est directe : la moyenne indique où se situe le capteur, la pente indique dans quelle direction et à quelle vitesse il se déplace. En pronostic, c'est la vitesse de dérive qui porte l'information sur l'imminence de la défaillance.

8.2.2. Le choix de la taille de fenêtre

La fenêtre de cinq cycles résulte d'un compromis. Une fenêtre plus courte serait insuffisamment lissante et laisserait le bruit dominer la tendance. Une fenêtre plus longue produirait une estimation plus stable mais introduirait une latence de détection accrue et sacrifierait davantage de cycles d'amorçage par moteur. Cinq cycles offrent un lissage suffisant tout en n'écartant que quatre lignes par unité, soit environ 1,8 % de l'ensemble d'entraînement.

8.2.3. Application uniforme aux vingt-et-un capteurs

Les statistiques glissantes sont appliquées aux vingt-et-un capteurs sans sélection préalable, y compris à ceux dont la variance est faible dans certains sous-ensembles. Ce choix diffère de la sélection restreinte à huit capteurs implémentée dans le modèle dbt. La justification est que le gradient boosting est intrinsèquement robuste aux variables peu informatives : l'algorithme ne les sélectionne simplement jamais lors de la construction des arbres. Filtrer manuellement introduirait un risque de rejeter un capteur informatif dans un sous-ensemble particulier au motif qu'il est constant dans un autre.

8.3. Protocole expérimental

Le protocole compare cinq modèles, entraînés et évalués dans des conditions strictement identiques.

Modèle	Variables	Rôle dans le protocole
mean_baseline	89	Référence triviale : prédit toujours la moyenne. Toute approche doit la dépasser largement
cycle_only_ridge	2	Teste combien d'information réside dans le seul numéro de cycle, sans lecture de capteurs
ridge_raw	26	Modèle linéaire sur capteurs bruts : mesure l'apport d'une relation linéaire simple
random_forest_raw	26	Modèle non linéaire sur capteurs bruts : isole l'apport de la non-linéarité
final_gradient_boosting	89	Modèle final : ajoute les variables de tendance à la non-linéarité

Table 14— Les cinq modèles du protocole comparatif.

La structure de cette comparaison est délibérément **incrémentale**. Chaque modèle isole une contribution : la référence à la moyenne mesure la variance de la cible ; le modèle à cycle unique mesure l'information triviale du temps ; le modèle linéaire mesure la part linéairement explicable ; la forêt aléatoire mesure l'apport de la non-linéarité à variables constantes ; et le modèle final mesure l'apport spécifique des variables de tendance. Cette conception permet d'attribuer la performance finale à des causes identifiées plutôt qu'à une boîte noire.

8.4. Le modèle final

Le modèle retenu est un [HistGradientBoostingRegressor](#) de scikit-learn, variante par histogrammes du gradient boosting. Cette implémentation discrétise les variables continues en un nombre fixe d'intervalles, ce qui réduit considérablement le coût de recherche des points de séparation et permet un entraînement sur 157 523 lignes et 89 variables en un temps

compatible avec une exécution dans une chaîne d'intégration continue.

Trois propriétés justifient ce choix pour des données tabulaires de cette nature :

- **Robustesse aux échelles hétérogènes** : les capteurs couvrent des ordres de grandeur très différents, des pressions de quelques unités aux températures de plusieurs centaines. Les modèles à base d'arbres sont invariants aux transformations monotones et ne requièrent donc aucune normalisation.
- **Capture native des interactions** : la dégradation d'un turboréacteur se manifeste par des combinaisons de signaux plutôt que par des seuils univariés. Les arbres successifs modélisent ces interactions sans spécification explicite.
- **Gestion des variables non informatives** : les variables de faible valeur prédictive sont simplement ignorées lors de la construction, sans dégradation de la performance.

8.5. Validation croisée par groupes

Une validation croisée à cinq plis, respectant les frontières entre moteurs, a été conduite sur l'ensemble d'entraînement. Ses résultats sont consignés dans le fichier de preuve final.

Pli	MAE	RMSE
1	11,692	16,795
2	11,504	16,927
3	11,451	16,821
4	11,353	16,705
5	11,158	16,217
Moyenne	11,432	16,693
Écart-type	0,177	0,258

Table 15— Résultats de la validation croisée à cinq plis par groupes de moteurs.

Deux enseignements se dégagent de ces chiffres. Premièrement, la stabilité est remarquable : l'écart-type de la MAE entre plis est inférieur à 0,18 cycle, soit environ 1,5 % de la valeur moyenne. Cette faible dispersion indique que la performance n'est pas tributaire d'un découpage favorable et que le modèle apprend un mécanisme généralisable plutôt que les particularités de sous-populations de moteurs.

Deuxièmement, l'écart entre la MAE de validation croisée (11,43) et la MAE de test standard (12,56) est faible, de l'ordre d'un cycle. Cette cohérence entre validation interne et évaluation externe constitue l'indicateur le plus fort de l'absence de surapprentissage et de l'efficacité des garde-fous anti-fuite décrits au chapitre précédent. Un écart important aurait au contraire signalé que le modèle exploitait une régularité propre à l'ensemble d'entraînement.

8.6. Chaîne d'entraînement et reproductibilité

L'entraînement est implémenté dans le script `scripts/train_model.py`, structuré en fonctions à responsabilité unique : chargement depuis DuckDB, calcul des variables de fenêtre, définition des métriques, construction de la chaîne de traitement, ajustement et évaluation, validation croisée, puis écriture des artefacts.

Une exigence forte a été posée quant à la preuve d'entraînement : le carnet de notes final n'est pas un document rédigé a posteriori, mais un carnet **réellement exécuté par un noyau Jupyter** via

le script [scripts/execute_training_notebook.py](#), et dont les sorties sont conservées dans [notebooks/training_executed.ipynb](#). Cette distinction importe : un carnet rédigé à la main peut afficher n'importe quel chiffre, un carnet exécuté porte la trace de son exécution.

Les artefacts produits par l'entraînement constituent le contrat de livraison vers l'équipe MLOps :

```
models/latest/model.pkl           # estimateur sérialisé
models/latest/feature_schema.json # liste ordonnée des 89 variables
models/latest/metrics.json       # métriques de l'exécution
reports/model_metrics/final_evaluation.json
reports/model_metrics/final_evaluation.md
notebooks/training_executed.ipynb
demo/predict_sample.json         # charge utile valide générée
```

Le fichier **feature_schema.json** mérite une mention particulière : il constitue le contrat exécutable entre l'entraînement et le service. L'API valide toute requête entrante contre cette liste et rejette les champs inconnus. Un décalage entre les variables attendues par le modèle et celles fournies par l'appelant devient ainsi une erreur immédiate et explicite plutôt qu'une prédiction silencieusement erronée.

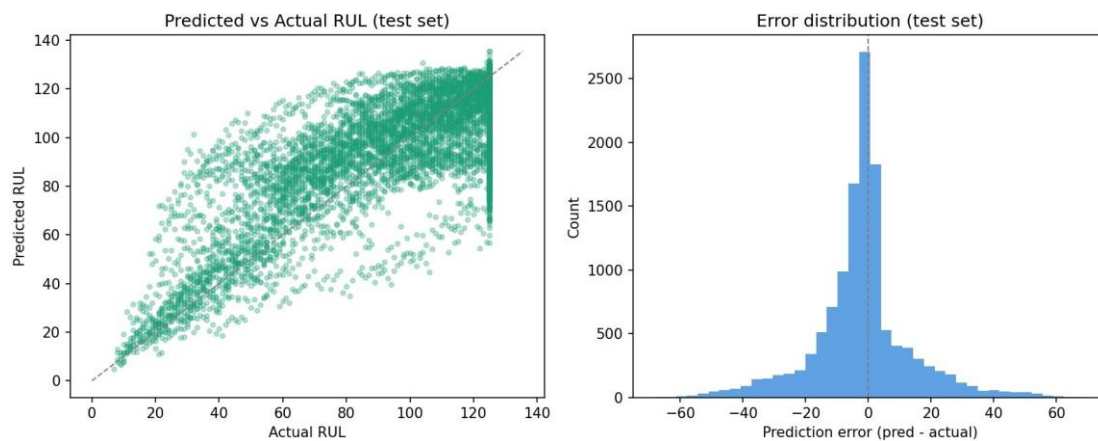


Figure 4— Évaluation du modèle amélioré : distribution des erreurs et adéquation prédiction/réalité.

CHAPITRE 9

Couche MLOps : suivi, registre et service de prédiction

9.1. Le suivi des expériences avec MLflow

Un projet d'apprentissage automatique produit un grand nombre d'exécutions dont seule une fraction sera retenue. Sans traçabilité systématique, la question « quels paramètres ont produit ce résultat ? » devient rapidement sans réponse. MLflow répond à ce besoin en enregistrant, pour chaque exécution, les paramètres, les métriques, les artefacts et l'identifiant unique de l'exécution.

Dans ce projet, chaque entraînement journalise les hyperparamètres du modèle, les métriques des cinq modèles comparés, le nombre de variables, la taille des jeux de données, ainsi que les artefacts : le modèle sérialisé, le schéma de variables, les rapports d'évaluation et la figure de comparaison. L'exécution finale porte l'identifiant [15c9d1d0e9f549a4a2b31d7e3b962f80](#).

9.2. Le registre de modèles

Le registre constitue la couche de gouvernance au-dessus du suivi. Il distingue explicitement l'exécution

un événement historique du **modèle enregistré** une entité versionnée et promouvable. Le modèle du projet est enregistré sous le nom [industrial-equipment-health-model](#).

Le mécanisme de promotion repose sur un **alias** plutôt que sur un numéro de version figé. À l'issue de chaque entraînement réussi, une nouvelle version est créée et l'alias champion est déplacé vers elle. Le service résout alors le modèle par l'URI suivante :

```
models:/industrial-equipment-health-model@champion
```

L'avantage opérationnel de ce mécanisme est considérable : le service n'a pas à connaître le numéro de version courant. Promouvoir un nouveau modèle ou revenir à une version antérieure se réduit à déplacer l'alias, sans modification de configuration ni redéploiement du service. C'est le fondement d'une stratégie de retour arrière rapide en cas d'incident.

Élément de gouvernance	Valeur du projet
Nom du modèle enregistré	industrial-equipment-health-model
Alias de promotion	champion
Version enregistrée	1
Identifiant d'exécution	15c9d1d0e9f549a4a2b31d7e3b962f80
Backend de suivi local	sqlite:///mlflow.db
Backend de suivi en pile Docker	http://mlflow:5000
URI de résolution par le service	models:/industrial-equipment-health-model@champion

Table 16 Paramètres de gouvernance du registre de modèles.

9.3. Le service de prédiction FastAPI

9.3.1. Le contrat de démarrage

Le choix de conception le plus significatif de la couche de service concerne son comportement au démarrage. Par défaut, **l'API refuse de démarrer si aucun modèle valide n'est disponible**. Un modèle de repli déterministe existe dans le code, mais il n'est activable que par la variable d'environnement explicite `ALLOW_FALLBACK_MODEL=true`, réservée aux tests unitaires.

La justification de ce choix est méthodologique. Un service qui, faute de modèle, répondrait par une valeur plausible produirait une démonstration parfaitement convaincante et scientifiquement nulle. L'échec visible au démarrage est préférable à la réussite trompeuse à l'exécution. Ce principe est explicitement inscrit dans les attentes opérationnelles du projet : un échec de chargement de modèle doit arrêter l'API plutôt que de servir de fausses prédictions.

9.3.2. Les points d'entrée principaux

Deux points d'entrée constituent le contrat public minimal.

Le point d'entrée `GET /health` retourne l'état du service et un ensemble d'informations de diagnostic : le statut, l'indicateur de chargement du modèle, la **source** du modèle — registre MLflow, fichier local, ou repli en test uniquement —, la version servie, le nombre de variables attendues, l'URI de suivi MLflow et le chemin du journal de surveillance. Ce point d'entrée ne se contente donc pas de répondre « je suis vivant » ; il répond « je suis vivant et voici exactement quel modèle je sers », ce qui le rend exploitable comme preuve d'audit.

Le point d'entrée `POST /predict` accepte une charge utile comportant le cycle en champ de premier niveau et l'ensemble des variables entraînées dans un objet `features`. La validation est stricte : tout champ inconnu provoque une réponse d'erreur 422. La réponse comprend l'identifiant du moteur, la RUL prédite, le niveau de risque, la version du modèle et la latence mesurée.

Champ de réponse	Type	Signification
engine_id	chaîne	Identifiant du moteur tel que fourni dans la requête
remaining_useful_life	réel	Durée de vie résiduelle prédite, en cycles
risk_level	chaîne	Niveau de risque dérivé par seuils : high, medium ou low
model_version	chaîne	Version du modèle ayant produit la prédiction
latency_ms	réel	Temps de traitement côté serveur, en millisecondes

Table 17— Structure de la réponse du point d'entrée de prédiction.

9.3.3. La dérivation du niveau de risque

Le niveau de risque est obtenu par application de deux seuils à la RUL prédite : le risque est qualifié d'élevé en deçà de trente cycles, de moyen entre trente et quatre-vingts cycles, et de faible au-delà. Ces seuils sont encapsulés dans une structure de données immuable, ce qui les rend explicites et modifiables en un point unique.

Il convient d'énoncer clairement la limite de cette approche : il ne s'agit pas d'un classifieur calibré. Les seuils n'ont pas été optimisés à partir d'une matrice de coûts de maintenance, et

la probabilité associée à chaque classe n'est pas estimée. Une industrialisation réelle exigerait une calibration fondée sur le coût économique d'une intervention anticipée comparé au coût d'une défaillance en service. Cette limite est reconnue et figure au chapitre des perspectives.

9.4. La console d'ingénierie et ses points d'entrée étendus

Au-delà du contrat public, le service expose un ensemble de points d'entrée sous le préfixe `/api`, destinés à la console d'ingénierie. Ils partagent le même service et le même modèle que le contrat public, sans génération de télémétrie aléatoire ni prédiction côté client.

Point d'entrée	Fonction
GET <code>/api/fleet/summary</code>	Répartition du risque sur la flotte et RUL agrégée prédite
GET <code>/api/fleet/engines</code>	Prédictions par moteur au dernier cycle, triables
GET <code>/api/engines?subset=FD004</code>	Moteurs et bornes de cycles d'un sous-ensemble
GET <code>/api/engines/{id}/cycles</code>	Bornes de rediffusion d'un moteur
GET <code>/api/engines/{id}/cycle/{cycle}</code>	Série historique des capteurs jusqu'au cycle sélectionné
POST <code>/api/engines/{id}/cycle/{cycle}/predict</code>	Prédiction réelle en mode opérations ou évaluation
POST <code>/api/predict/batch</code>	Validation et scoring d'un fichier CSV de format C-MAPSS
GET <code>/api/model/info</code>	Identité du modèle et preuves d'évaluation sur données retenues
GET <code>/api/platform/status</code>	Preuves d'artefacts DataOps et MLOps
GET <code>/api/monitoring/summary</code>	Volume de prédictions, latence, risque, évolution et dérive
POST <code>/api/maintenance/plan</code>	Recommandation de maintenance issue d'une prédiction réelle

Table 18 point d'entrée de consol

Deux propriétés de conception méritent d'être relevées. D'une part, la rediffusion d'un moteur exige au minimum cinq cycles observés, contrainte directement héritée du schéma de variables comportant des statistiques glissantes sur cinq cycles : l'API ne peut pas produire une prédiction dont les variables ne seraient pas intégralement définies. D'autre part, le mode opérations ne retourne jamais la vérité future, tandis que le mode évaluation peut exposer la RUL réelle et l'erreur de prédiction, les étiquettes de test C-MAPSS étant disponibles pour l'évaluation hors ligne.

Cette séparation stricte des deux modes constitue une garantie méthodologique importante : elle rend structurellement impossible qu'une démonstration en mode opérations bénéficie, même involontairement, d'une information indisponible en exploitation réelle.

CHAPITRE 10

Conteneurisation, intégration continue et surveillance

10.1. La reproductibilité comme exigence première

La question centrale de l'industrialisation peut se formuler simplement : un tiers, disposant du seul dépôt, peut-il reproduire le résultat annoncé sur sa propre machine ? Toute réponse négative invalide l'ensemble des affirmations de performance, puisqu'elles deviennent invérifiables. La conteneurisation est le moyen technique de rendre cette réponse positive.

L'objectif retenu est une exécution en une commande unique :

```
docker compose up --build
```

Cette commande démarre le serveur MLflow, exécute le service d'initialisation qui télécharge les données et matérialise l'intégralité du graphe Dagster — ingestion, transformation, tests, entraînement réel, enregistrement du modèle, preuves de surveillance —, puis démarre le service de prédiction et l'interface Dagster. L'évaluateur n'obtient donc pas une démonstration figée mais une reconstruction complète effectuée sous ses yeux.

10.2. Conception des images et de la pile

Un unique fichier [Dockerfile](#) construit une image partagée par tous les services, différenciés par leur commande de démarrage et leurs variables d'environnement. Ce choix réduit la surface de maintenance et garantit que tous les services s'exécutent exactement dans le même environnement Python, éliminant une source classique d'écart entre entraînement et service.

La configuration s'appuie sur des ancres YAML définissant un bloc de variables d'environnement commun : chemin du code source, répertoire des données brutes, chemin de la base DuckDB, URI de suivi MLflow, nom et alias du modèle, chemin du modèle et du schéma de variables, chemin du journal de prédiction et liste des sous-ensembles à entraîner.

Volume	Rôle	Partagé entre
./data	Fichiers bruts téléchargés	training-init
./models	Artefacts de modèle de récupération	training-init, api
./logs	Journal des prédictions en JSONL	api
./reports	Rapports d'évaluation et de dérive	training-init, api
./warehouse	Base DuckDB	training-init, api, dagster
mlflow-storage	Base de suivi et dossier d'artefacts MLflow	tous les services
dagster-storage	État de matérialisation des actifs	training-init, dagster

Table 19 — Volumes persistants de la pile Docker Compose.

Les dépendances entre services sont exprimées par des conditions de santé plutôt que par de simples ordres de démarrage : le service d'initialisation attend que MLflow soit effectivement opérationnel, et non seulement démarré. Le contrôle de santé interroge le point d'entrée dédié

du serveur avec une période de démarrage étendue à cinq minutes, ce qui absorbe le temps d'initialisation de la base de suivi lors du premier lancement.

10.3. La chaîne d'intégration continue

La chaîne d'intégration continue, définie dans [.github/workflows/ci.yml](https://github.com/Workflows/ci.yml), se déclenche sur chaque demande de fusion et sur chaque poussée vers la branche principale ou la branche d'intégration. Elle se distingue de la plupart des chaînes de projets étudiants par un choix fort : elle ne se contente pas de vérifier la syntaxe, elle rejoue la totalité du flux.

Étape	Action	Nature de la garantie
1	Récupération du dépôt et installation de Python 3.12	Environnement reproductible
2	Installation des dépendances depuis requirements.txt	Dépendances déclarées et résolues
3	Téléchargement des données NASA C-MAPSS	Source accessible et automatisable
4	Exécution de dlt, dbt et des contrats de données	Chaîne DataOps fonctionnelle et tests de qualité réussis
5	Entraînement et enregistrement d'un modèle d'intégration	La chaîne d'entraînement s'exécute réellement
6	Tests unitaires et tests DataOps	Régressions logicielles détectées
7	Compilation, analyse statique ruff, contrôle de disponibilité, validation de la configuration Compose	Qualité statique et configuration valide
8	Construction des images de conteneurs	Les images se construisent effectivement
9	Démarrage de MLflow, exécution de l'initialisation, démarrage de l'API, test de fumée de prédiction	Le service répond réellement à une requête de prédiction
10	Nettoyage systématique des conteneurs	Absence d'effet de bord entre exécutions

Table 20— Les dix étapes de la chaîne d'intégration continue.

L'étape neuf constitue l'apport le plus significatif. Un test de fumée qui démarre la pile, exécute une prédiction réelle et valide la structure de la réponse garantit que la promesse principale du projet un service qui prédit est vérifiée automatiquement à chaque modification. Une régression qui casserait le chargement du modèle ou le contrat de l'API serait détectée immédiatement, et non lors de la démonstration finale.

Une optimisation pragmatique mérite d'être signalée : la chaîne d'intégration s'exécute sur le seul sous-ensemble FD001, configuré par la variable `TRAIN_SUBSETS`. L'objectif de l'intégration continue est de vérifier que le flux fonctionne, non de reproduire l'entraînement complet, qui serait excessivement coûteux en temps de calcul partagé. L'entraînement sur les quatre sous-ensembles demeure le mode par défaut de la pile Docker locale.

10.4. Stratégie de tests

Le dépôt comporte dix fichiers de tests couvrant des périmètres distincts, pour un total d'environ huit cents lignes de code de test.

Fichier de test	Périmètre couvert
test_dataops.py	Chaîne d'ingestion, tables DuckDB, modèles et tests dbt
test_maaskk_api_app.py	Comportement du service : démarrage, santé, validation stricte, rejet des champs inconnus
test_maaskk_mlops_contracts.py	Contrats de la couche MLOps : chargement, métadonnées, résolution d'URI
test_maaskk_monitoring_and_loader.py	Journalisation des prédictions, calcul de dérive, chargeur de modèle
test_maaskk_scripts_and_readiness.py	Scripts utilitaires et rapport de disponibilité
test_frontend_data_service.py	Service de données de la console : accès aux trajectoires, construction des vecteurs
test_frontend_proxy.py	Relais de l'interface publique vers le service géré
test_production_delivery.py	Présence et cohérence des artefacts de livraison
test_service_info.py	Informations d'identification du service
test_scaffold.py	Vérification structurelle du dépôt

Table 21— Couverture de la stratégie de tests.

10.5. Surveillance et observabilité

10.5.1. Les signaux surveillés

La couche de surveillance a été délibérément conçue comme simple et explicable, avec une trajectoire de remplacement identifiée vers Prometheus, Grafana ou Evidently.

Signal	Intérêt opérationnel	Implémentation
Disponibilité du service	Prouve que l'API est vivante et a chargé un modèle réel	GET /health avec statut, source et nombre de variables
Latence de réponse	Mesure la performance de service	Champ latency_ms dans chaque réponse
Journal de prédictions	Crée la traçabilité des décisions	Fichier logs/prediction_logs.jsonl
Version de modèle	Atteste de la discipline de registre	Champ model_version dans la santé et les prédictions
Dérive simple	Détecte un déplacement de la distribution des prédictions	Script de génération de rapport de dérive

Table 22— Signaux de surveillance et leur implémentation.

10.5.2. Le journal de prédictions

Chaque appel de prédiction ajoute une ligne au format JSONL comportant l'horodatage UTC, l'identifiant du moteur, la RUL prédite, le niveau de risque, la version du modèle et la latence mesurée. Le choix du format ligne à ligne n'est pas anodin : il permet l'ajout en fin de fichier sans relecture, il résiste à une interruption brutale une ligne incomplète n'invalide pas les précédentes et il se prête directement à une ingestion ultérieure par un outil d'analyse.

10.5.3. La détection de dérive

Le module de dérive implémente une comparaison volontairement élémentaire : il calcule la moyenne et l'écart-type d'une variable numérique sur une période de référence et sur une période courante, puis signale une dérive lorsque le déplacement absolu de la moyenne dépasse un seuil configurable. Le script de génération divise le journal de prédictions en deux moitiés, la première servant de référence à la seconde.

Le rapport produit lors de la répétition finale illustre ce fonctionnement :

Grandeur	Valeur	Interprétation
Variable surveillée	<code>remaining_useful_life</code>	Distribution des sorties du modèle
Moyenne de référence	106,25	Première moitié de la fenêtre d'observation
Moyenne courante	67,75	Seconde moitié
Déplacement absolu	38,50	Supérieur au seuil configuré
Écart-type de référence	9,60	Dispersion initiale
Écart-type courant	5,93	Resserrement de la distribution
Dérive détectée	oui	Déclenchement du signal

Table 23— Exemple de rapport de dérive généré durant la répétition.

Trois points doivent être énoncés en toute transparence. Premièrement, ce rapport particulier a été produit à partir de valeurs de démonstration documentées, le journal de prédictions comportant alors moins de quatre lignes ; le champ `source` du fichier JSON l'indique explicitement, ce qui constitue précisément la forme d'honnêteté instrumentale que le projet revendique. Deuxièmement, la méthode ne détecte que les déplacements de moyenne et resterait aveugle à un changement de forme de distribution à moyenne constante. Troisièmement, elle surveille la distribution des prédictions et non celle des variables d'entrée, ce qui constitue un indicateur retardé.

Ces limites sont assumées au regard de l'objectif pédagogique : disposer d'un mécanisme entièrement compréhensible et explicable en démonstration, plutôt que d'une bibliothèque tierce dont le fonctionnement interne resterait opaque à l'équipe.

10.6. Le rapport de disponibilité

Un script de vérification de disponibilité produit un rapport structuré recensant l'état des conditions nécessaires à une démonstration valide : présence du modèle et du schéma de variables, accessibilité du suivi MLflow, existence des rapports d'évaluation et de dérive. Ce rapport constitue une liste de contrôle exécutable plutôt qu'une vérification manuelle, conformément au principe général du projet selon lequel toute vérification répétée doit être automatisée.

CHAPITRE 11

Résultats et discussion

11.1. Résultats comparatifs des modèles

Le tableau suivant rassemble les performances des cinq modèles selon le protocole standard du dernier cycle observé, évalué sur 707 moteurs de test. Il s'agit de la métrique de référence du projet.

Modèle	Variables	MAE	RMSE	Score NASA
mean_baseline	89	37,315	44,445	351 372
cycle_only_ridge	2	29,848	36,999	140 147
ridge_raw	26	18,832	23,046	11 194
random_forest_raw	26	12,727	17,287	4 813
final_gradient_boosting	89	12,557	16,925	4 466

Table 24— Performances comparées selon le protocole standard du dernier cycle (707 moteurs).

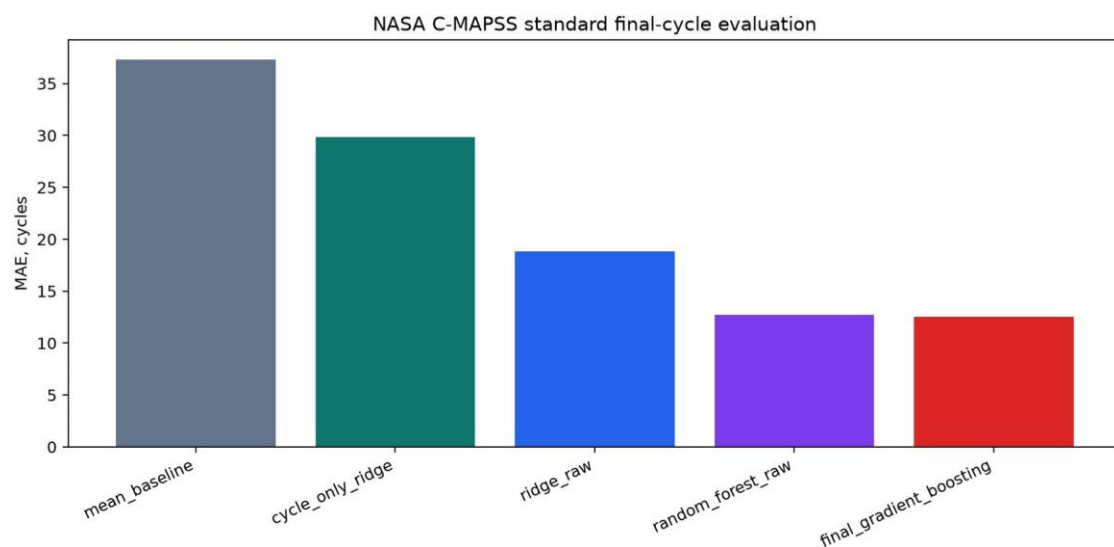


Figure 5— Comparaison graphique des cinq modèles évalués.

11.2. Lecture incrémentale des gains

La structure incrémentale du protocole permet d'attribuer la performance à des causes identifiées. Le tableau suivant décompose la contribution de chaque saut méthodologique.

Transition	Gain de MAE	Gain relatif	Ce que le gain démontre
Moyenne → cycle seul	7,47	20,0 %	Le numéro de cycle porte à lui seul une information substantielle sur l'usure
Cycle seul → Ridge brut	11,02	36,9 %	La lecture des capteurs apporte davantage que le simple écoulement du temps
Ridge brut → forêt aléatoire	6,11	32,4 %	La relation entre capteurs et dégradation est fortement non linéaire
Forêt aléatoire → boosting avec tendances	0,17	1,3 %	L'apport marginal des variables de tendance est faible sur cette métrique
Total : moyenne → modèle final	24,76	66,3 %	Réduction des deux tiers de l'erreur par rapport à la référence triviale

Table 25 — Décomposition incrémentale des gains de performance.

La lecture de ce tableau conduit à une conclusion nuancée qu'il serait malhonnête de dissimuler. Le gain le plus important ne provient pas du modèle final mais du **passage à un modèle non linéaire sur capteurs bruts** : la forêt aléatoire atteint déjà 12,73 de MAE avec seulement 26 variables. L'ajout de 63 variables de tendance et le passage au gradient boosting n'apportent qu'un gain marginal de 0,17 cycle sur la métrique du dernier cycle, soit 1,3 %.

Le modèle final est meilleur, mais pas spectaculairement meilleur qu'une forêt aléatoire sur capteurs bruts. Reconnaître cela est plus utile scientifiquement que de présenter le gain de 66 % sur la référence triviale comme si l'ingénierie des variables en était seule responsable

Ce résultat appelle toutefois deux nuances importantes. D'une part, l'écart est plus marqué sur le score NASA, 4 466 contre 4 813, soit une réduction de 7,2 % — ce qui indique que le modèle final commet **moins de surestimations dangereuses** de la durée de vie, propriété opérationnellement décisive même à MAE quasi identique. D'autre part, le modèle final présente un temps d'inférence et une empreinte mémoire nettement inférieurs à ceux d'une forêt aléatoire de taille comparable, ce qui pèse dans un contexte de service à faible latence.

11.3. Analyse des deux protocoles d'évaluation

Le tableau suivant confronte les deux protocoles pour le modèle final.

Protocole	Observations	MAE	RMSE	Score NASA
Dernier cycle observé (standard)	707	12,557	16,925	4 466
En ligne, au niveau de la ligne	102 069	10,190	15,738	781 814

Table 26 — Comparaison des deux protocoles d'évaluation pour le modèle final.

Deux observations appellent explication. D'abord, la MAE en ligne est **inférieure** à la MAE standard, ce qui peut sembler paradoxal. L'explication tient à la composition des observations

: l'évaluation en ligne porte sur l'ensemble des cycles de chaque trajectoire, incluant de nombreux cycles de début de vie où la RUL est écrêtée à 125 et donc triviale à prédire. L'évaluation standard, elle, ne porte que sur le dernier cycle observé, c'est-à-dire précisément la zone où la prédiction est la plus difficile et la plus utile. La métrique standard est donc à la fois plus exigeante et plus pertinente.

Ensuite, l'écart d'ordre de grandeur des scores NASA — 4 466 contre 781 814 — ne traduit aucune anomalie. Le score NASA est une **somme** de pénalités exponentielles ; sa valeur croît mécaniquement avec le nombre d'observations. Comparer un score sur 707 observations à un score sur 102 069 observations n'a aucun sens. Cette propriété justifie que la métrique standard ait été retenue comme référence unique du projet et que la métrique en ligne soit explicitement qualifiée de preuve secondaire.

11.4. Cohérence entre validation et test

La confrontation entre validation croisée et évaluation finale constitue le test de robustesse le plus significatif.

Source de l'estimation	MAE	RMSE	Écart à la référence
Validation croisée, moyenne des 5 plis	11,432	16,693	—
Test standard, dernier cycle	12,557	16,925	+1,125 en MAE

Table 27— Cohérence entre validation croisée interne et évaluation externe.

L'écart de 1,13 cycle en MAE est faible et va dans le sens attendu : la performance de test est légèrement inférieure à la performance de validation, ce qui est normal puisque le test porte exclusivement sur les cycles terminaux les plus difficiles. L'absence d'écart important est l'indicateur recherché : un modèle surajusté afficherait typiquement une validation croisée très optimiste et un test dégradé. La faible dispersion entre plis, avec un écart-type de 0,18, renforce cette conclusion.

11.5. Interprétation opérationnelle des résultats

Une MAE de 12,56 cycles signifie que la prédiction se trompe en moyenne d'un peu plus de douze cycles de fonctionnement. Que vaut ce chiffre en pratique ?

Rapporté à la durée de vie typique d'une unité C-MAPSS, comprise entre 130 et 350 cycles, une erreur de douze cycles représente entre 3,5 % et 9,5 % de la vie totale. Comparativement, une politique de maintenance systématique calibrée pour garantir la sécurité devrait intervenir à la durée de vie minimale observée sur la flotte, gaspillant ainsi la totalité de la marge des unités plus robustes — soit potentiellement plus de deux cents cycles pour les meilleures d'entre elles.

La valeur ajoutée du modèle est donc considérable dans son principe : il permet de resserrer la fenêtre de décision de plusieurs centaines de cycles à quelques dizaines. Il faut cependant immédiatement en tempérer la portée : **une erreur moyenne n'est pas une garantie**. La RMSE de 16,93, supérieure de 35 % à la MAE, indique la présence d'erreurs individuelles nettement plus importantes que la moyenne. Une politique de maintenance responsable devrait donc appliquer une marge de sécurité dérivée du quantile supérieur de la distribution des erreurs, et non de sa moyenne.

11.6. Résultats de la chaîne DataOps et d'industrialisation

Au-delà des performances prédictives, la table suivante récapitule les résultats vérifiés de la chaîne d'industrialisation, qui constituent l'apport principal du projet au regard des objectifs du module.

Critère	Résultat vérifié
Ingestion reproductible des données brutes	dlt charge l'intégralité de FD001 à FD004, paquet sans travail en échec
Entrepôt en couches	Schémas raw, staging et marts présents dans DuckDB
Transformations déclaratives	3 modèles dbt construits, aucune transformation métier en SQL brut hors dbt
Tests de qualité	8 tests dbt réussis sur 8
Tests logiciels	11 tests DataOps réussis, 10 fichiers de tests au total
Orchestration	7 actifs Dagster dans un graphe unique, planification quotidienne
Suivi d'expériences	Paramètres, métriques et artefacts journalisés dans MLflow
Registre de modèles	Version 1 enregistrée, alias champion positionné
Service de prédiction	/health et /predict opérationnels, validation stricte du schéma
Reproductibilité	Pile complète démarrée par une commande unique
Intégration continue	10 étapes incluant entraînement, construction et test de fumée
Surveillance	Journal JSONL par prédiction et rapport de dérive généré

Table 28— Synthèse des résultats d'industrialisation vérifiés.

11.7. Discussion critique

11.7.1. Ce qui a bien fonctionné

Le déplacement de la logique de transformation vers dbt s'est révélé être la décision technique la plus rentable du projet. Elle a rendu la transformation lisible, testable et traçable, et a permis de placer la barrière de qualité en amont de l'entraînement dans le graphe d'orchestration. Le refus du repli silencieux dans le service a également prouvé sa valeur : il a contraint l'équipe à disposer d'un modèle réellement entraîné avant toute démonstration, éliminant la tentation d'une façade fonctionnelle.

11.7.2. Ce qui s'est révélé plus difficile que prévu

L'intégration des contributions a constitué la difficulté majeure, non pour des raisons techniques mais organisationnelles. Le modèle « une branche par personne » a produit l'autonomie recherchée au prix d'une dette d'intégration concentrée en fin de projet. Deux périmètres n'ayant pas produit de livrable exploitable, l'intégration a dû reconstruire les documents d'API, les charges utiles de démonstration et les artefacts agiles, ce qui a déplacé la charge vers un nombre restreint de contributeurs.

11.7.3. Les écarts entre conception et implémentation

Trois écarts ont été identifiés et sont explicitement assumés. Le document de lignage décrit une couche intermédiaire à trois modèles qui n'a pas été implémentée telle quelle, les transformations ayant été consolidées. La stratégie de qualité documente des contrôles de plages et de monotonie qui relèvent de la feuille de route et non de l'implémentation actuelle, limitée à huit tests structurants. Le modèle dbt applique les statistiques glissantes à huit capteurs sélectionnés, alors que la chaîne d'entraînement Python les applique aux

vingt-et-un capteurs, ce qui introduit une redondance partielle entre les deux couches.

Ce dernier point mérite une remarque architecturale. Idéalement, l'ingénierie des variables devrait résider en un lieu unique. La situation actuelle, où une partie des variables est calculée en SQL et une autre en Python, est fonctionnelle mais constitue une dette : une modification de la définition d'une fenêtre devrait être répercutée en deux endroits. La consolidation de l'ensemble des variables dans la couche dbt constitue la première recommandation technique de la feuille de route.

CHAPITRE 12

Limites et perspectives

12.1. Limites liées aux données

La limite la plus fondamentale est la nature simulée du jeu de données. C-MAPSS est produit par un modèle et non par une flotte réelle. Il en résulte l'absence de plusieurs phénomènes qui dominent l'exploitation industrielle réelle : valeurs manquantes accidentelles, pannes de capteurs, dérives de calibration, interruptions de télémétrie, interventions de maintenance intermédiaires modifiant la trajectoire de dégradation, et hétérogénéité de qualité entre unités d'instrumentation.

Une conséquence pratique doit être soulignée : les mécanismes de qualité des données mis en place dans ce projet, bien que méthodologiquement corrects, n'ont **jamais été confrontés à des données réellement dégradées**. Les huit tests dbt passent en permanence, ce qui est rassurant mais aussi peu informatif quant à leur capacité de détection réelle. Un test qui n'a jamais échoué n'a pas démontré son efficacité.

Par ailleurs, la vérité terrain de la RUL n'est disponible en exploitation réelle qu'après la défaillance ou la dépose, soit potentiellement plusieurs mois après la prédiction. Toute surveillance fondée sur l'erreur réelle serait donc structurellement en retard, ce qui justifie l'approche par surveillance de la distribution des prédictions retenue ici, tout en en soulignant le caractère de substitut.

12.2. Limites liées à la modélisation

- **Le niveau de risque n'est pas calibré.** Les seuils de trente et quatre-vingts cycles sont fixés a priori, sans référence à une matrice de coûts de maintenance ni estimation de probabilité par classe.
- **L'incertitude de prédiction n'est pas quantifiée.** Le modèle retourne une valeur ponctuelle sans intervalle de confiance, alors qu'une décision de maintenance responsable devrait s'appuyer sur une borne inférieure prudente plutôt que sur une estimation centrale.
- **L'interprétabilité est limitée.** Aucune analyse d'importance de variables ni d'attribution locale par valeurs de Shapley n'a été conduite, ce qui restreint la capacité à expliquer une prédiction individuelle à un ingénieur de maintenance.
- **Aucune architecture séquentielle n'a été évaluée.** Les réseaux récurrents à mémoire longue et les architectures à convolutions temporelles obtiennent des performances supérieures dans la littérature C-MAPSS, mais au prix d'un coût de calcul et d'une complexité d'industrialisation incompatibles avec le périmètre du projet.

12.3. Limites liées à l'industrialisation

Le déploiement est local. Bien qu'autorisé explicitement par l'encadrement, il ne confronte pas le projet aux problématiques réelles d'un environnement de production distribué : montée

en charge, haute disponibilité, gestion des secrets, isolation réseau, sauvegarde et restauration, et coût d'exploitation.

La détection de dérive, conçue pour être explicable, ne compare que des moyennes et resterait aveugle à un changement de forme de distribution à moyenne constante. Elle porte en outre sur les sorties du modèle et non sur ses entrées, ce qui en fait un indicateur retardé.

Enfin, aucun mécanisme de réentraînement automatique n'est implémenté : la détection d'une dérive produit un rapport, mais aucune action corrective n'en découle automatiquement. Le projet se situe donc au niveau de maturité MLOps correspondant à une chaîne automatisée avec intégration continue, sans atteindre le niveau de la boucle fermée.

12.4. Perspectives d'évolution

12.4.1. Court terme

Action	Bénéfice attendu	Effort
Consolider toutes les variables explicatives dans la couche dbt	Suppression de la redondance SQL/Python et point de vérité unique	Modéré
Remplacer le lignage manuel par la documentation générée par dbt	Élimination de la divergence entre documentation et code	Faible
Implémenter les tests de plages et de monotonie de la RUL	Passage d'une couverture structurelle à une couverture sémantique	Faible
Ajouter une analyse d'importance des variables et des valeurs de Shapley	Explicabilité des prédictions individuelles	Modéré
Calibrer les seuils de risque sur une matrice de coûts	Décision de maintenance fondée sur l'économie réelle	Modéré

Table 29— Perspectives à court terme.

12.4.2. Moyen terme

- Substituer à la détection de dérive artisanale une bibliothèque spécialisée telle qu'Evidently, permettant la surveillance conjointe des entrées et des sorties avec des tests statistiques établis.
- Exposer les métriques de service au format Prometheus et construire un tableau de bord Grafana couvrant latence, volume, répartition du risque et indicateurs de dérive.
- Quantifier l'incertitude par régression quantile ou par ensemble, afin de fournir une borne inférieure de RUL exploitable en planification prudente.
- Introduire un mécanisme de réentraînement déclenché par la dérive, avec promotion conditionnée à la validation sur un jeu de référence figé.

12.4.3. Long terme

À plus longue échéance, trois directions se dessinent. Le déploiement sur une infrastructure gérée permettrait de confronter la plateforme aux exigences réelles de haute disponibilité et de montée en charge. L'évaluation d'architectures séquentielles profondes, dans un cadre de comparaison honnête tenant compte du coût d'inférence, permettrait de situer le compromis performance/industrialisabilité. Enfin, la confrontation de la chaîne à

des données industrielles réelles constituerait le véritable test de validité des mécanismes de qualité, puisqu'elle exposerait pour la première fois les tests à des données susceptibles de les faire échouer.

Conclusion générale

Ce projet avait pour objet la conception et l'industrialisation d'une plateforme complète de maintenance prédictive, appliquée à l'estimation de la durée de vie résiduelle de turboréacteurs à partir du jeu de données NASA C-MAPSS. Il s'inscrivait dans un module consacré aux pratiques MLOps et DataOps, dont l'exigence centrale n'était pas la performance algorithmique mais la maîtrise du cycle de vie complet d'un produit de données.

Sur le plan technique, les objectifs fixés ont été atteints. La chaîne DataOps ingère les quatre sous-ensembles C-MAPSS via dlt, les stocke dans un entrepôt DuckDB organisé en trois couches, les transforme par des modèles dbt déclaratifs et versionnés, les soumet à huit tests de qualité bloquants, et orchestre l'ensemble dans un graphe Dagster de sept actifs dont l'entraînement dépend structurellement de la réussite des tests. La chaîne MLOps entraîne un modèle de gradient boosting sur 89 variables explicatives, journalise chaque exécution dans MLflow, enregistre le modèle dans un registre avec alias de promotion, et sert ce modèle exact via FastAPI sans possibilité de repli silencieux. L'ensemble se démarre par une commande unique et est vérifié à chaque modification par une chaîne d'intégration continue en dix étapes incluant un test de fumée de prédiction réelle.

Sur le plan des performances, le modèle final atteint une erreur absolue moyenne de 12,56 cycles et une racine d'erreur quadratique moyenne de 16,93 cycles selon le protocole standard du dernier cycle observé, sur 707 moteurs de test couvrant les quatre sous-ensembles simultanément. Ce résultat représente une réduction de 66 % de l'erreur absolue par rapport à une référence triviale. La cohérence entre validation croisée par groupes et évaluation finale, avec un écart inférieur à 1,2 cycle, atteste de l'absence de surapprentissage et de l'efficacité des garde-fous anti-fuite mis en place.

L'analyse critique conduite au chapitre 11 tempère toutefois l'attribution de ce gain. La part prépondérante de la performance provient du passage à un modèle non linéaire sur capteurs bruts, tandis que l'apport marginal des soixante-trois variables de tendance reste modeste sur la métrique de référence, bien qu'il se traduise par une réduction significative des surestimations dangereuses au sens du score asymétrique. Reconnaître cette nuance relève de la même exigence d'honnêteté qui a présidé au refus du repli silencieux dans le service.

La contribution principale de ce travail ne réside donc pas dans ses performances prédictives, qui demeurent en deçà de l'état de l'art algorithmique sur C-MAPSS, mais dans la démonstration qu'une performance honnête et vérifiable peut être obtenue à l'intérieur d'une chaîne entièrement reproductible, testée, versionnée et surveillée. Chaque chiffre avancé dans ce rapport est rattachable à un artefact du dépôt

: un fichier de métriques en JSON, un carnet réellement exécuté par un noyau Jupyter, un rapport de dérive horodaté. Cette traçabilité constitue, dans un domaine où la reproductibilité fait défaut de manière chronique, un résultat en soi.

Les enseignements organisationnels ne sont pas moindres. Le modèle de travail par branches indépendantes a produit l'autonomie recherchée mais a concentré la dette d'intégration en fin de projet. L'expérience confirme que l'intégration continue est d'abord une discipline collective avant d'être un outil : une chaîne automatisée ne produit de valeur

que si les contributions y sont soumises fréquemment. Cette leçon, consignée dans la rétrospective finale, constitue probablement l'apport le plus transférable du projet.

Les perspectives identifiées consolidation de l'ingénierie des variables dans une couche unique, documentation de lignage générée depuis le code, quantification de l'incertitude, calibration économique des seuils de risque, surveillance conjointe des entrées et des sorties, et boucle de réentraînement dessinent une trajectoire cohérente vers un niveau de maturité supérieur. Elles supposent moins des ruptures technologiques que la poursuite méthodique de la même discipline : rendre explicite ce qui est implicite, automatiser ce qui est répété, et prouver ce qui est affirmé.

Bibliographie et sources

Jeu de données

6. NASA Ames Prognostics Center of Excellence, *Turbofan Engine Degradation Simulation Data Set*, Prognostics Data Repository, National Aeronautics and Space Administration. Source primaire des données d'entraînement et de test du projet.
7. A. Saxena, K. Goebel, D. Simon et N. Eklund, « Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation », *International Conference on Prognostics and Health Management*, 2008. Publication de référence décrivant la génération du jeu C-MAPSS et la métrique de score asymétrique.

Pronostic et maintenance prédictive

8. A. Heng, S. Zhang, A. C. C. Tan et J. Mathew, « Rotating machinery prognostics: State of the art, challenges and opportunities », *Mechanical Systems and Signal Processing*, 2009.
20. X.-S. Si, W. Wang, C.-H. Hu et D.-H. Zhou, « Remaining useful life estimation — A review on the statistical data driven approaches », *European Journal of Operational Research*, 2011.
21. R. K. Mobley, *An Introduction to Predictive Maintenance*, 2e édition, Butterworth-Heinemann, 2002. Cadre général des stratégies de maintenance corrective, préventive et conditionnelle.

MLOps, DataOps et dette technique

22. D. Sculley et al., « Hidden Technical Debt in Machine Learning Systems », *Advances in Neural Information Processing Systems*, 2015. Source du constat sur la proportion minoritaire du code d'apprentissage dans un système réel.
23. E. Breck, S. Cai, E. Nielsen, M. Salib et D. Sculley, « The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction », *IEEE International Conference on Big Data*, 2017.
24. M. Treveil et l'équipe Dataiku, *Introducing MLOps: How to Scale Machine Learning in the Enterprise*, O'Reilly Media, 2020.
25. C. Huyen, *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*, O'Reilly Media, 2022.
26. J. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017. Cadre conceptuel pour les architectures de données en couches.

Documentation technique des outils

27. dltHub, *dlt — data load tool*, documentation officielle de la bibliothèque d'ingestion Python.
28. DuckDB Foundation, *DuckDB — An in-process SQL OLAP database management system*, documentation officielle.
29. dbt Labs, *dbt — data build tool*, documentation officielle des modèles, tests et documentation générée.
30. Dagster Labs, *Dagster — a data orchestrator for machine learning, analytics and ETL*, documentation officielle du modèle orienté actifs.
31. MLflow Project, *MLflow Documentation — Tracking, Models and Model Registry*, Linux Foundation.
32. S. Ramírez, *FastAPI Documentation*, référence du cadriciel de service et de la validation par Pydantic.
33. F. Pedregosa et al., « Scikit-learn: Machine Learning in Python », *Journal of Machine Learning Research*, vol. 12, 2011.
34. Docker Inc., *Docker Compose Specification*, documentation officielle de la composition de services.

Artefacts internes du projet

35. [README.md](#) — vue d'ensemble, architecture cible, commandes de lancement et preuves d'entraînement.
36. [docs/final-report/final-report.md](#) — rapport de synthèse technique du dépôt.
37. [docs/architecture.md](#), [docs/dataops.md](#), [docs/data-quality.md](#), [docs/data-lineage.md](#) — documentation d'architecture et de gouvernance.
38. [docs/api.md](#), [docs/monitoring.md](#), [docs/mlflow-model-registry.md](#), [docs/mlops-runbook.md](#) — documentation d'exploitation.
39. [agile/product-backlog.md](#), [agile/sprint-plan.md](#), [agile/sprint-review-retro.md](#) — artefacts de gestion de projet.
40. [reports/model_metrics/final_evaluation.json](#) et [.md](#) — preuves de métriques de l'exécution finale.
41. [reports/monitoring/drift_report.json](#) — preuve de surveillance de dérive.
42. [notebooks/training_executed.ipynb](#) — carnet d'entraînement réellement exécuté.